

The C Programming Language

From Beginner to Expert

A two-part tutorial with worked examples from a real embedded machine-learning codebase

Part I	General-Purpose C (Chapters 1-16)
Part II	Embedded & Systems C (Chapters 17-28)
Appendix A	Full Exercise Solutions
Appendix B	Quick Reference Tables
Code from	ODT - On-Device Training library, STM32 Nucleo-F746ZG
Application	UCI HAR - Human Activity Recognition, 6 classes, 9-channel IMU

How to Use This Book

This book teaches C twice over. Part I builds the language from nothing: variables, control flow, functions, pointers, memory, the standard library. Part II takes that same language and turns it toward embedded systems: bit manipulation, fixed-point arithmetic, memory that must fit in 320 KB, and code that runs on a microcontroller with no operating system underneath it.

Every concept is introduced twice. First with a small, self-contained example you can compile and run on any computer. Then, wherever it applies, with the same concept as it actually appears in ODT — a real on-device neural network training library written in pure C for the STM32 Nucleo-F746ZG. Those second examples are marked with a purple bar.

What ODT is

ODT (On-Device Training) trains a small neural network directly on a microcontroller. It has no operating system, no dynamic memory during training, and 320 KB of RAM total. Every technique in Part II exists in ODT because the constraints forced it there. When you see a purple box, you are looking at C that has to work under real pressure.

Compile everything

Reading C teaches you very little. Type the examples in, compile them, break them on purpose, and see what the compiler says. Use `gcc -Wall -Wextra -std=c11 file.c -o prog` so the compiler warns you about the mistakes it can see.

Exercises

Every chapter ends with exercises in a yellow box. Full worked solutions are in Appendix A. Attempt them before looking — the solutions explain the reasoning, not just the answer.

Contents

PART I **GENERAL-PURPOSE C**

- | | |
|----|-------------------------------------|
| 1 | Getting Started: Your First Program |
| 2 | Variables, Types and Memory |
| 3 | Operators and Expressions |
| 4 | Control Flow: Branching |
| 5 | Control Flow: Loops |
| 6 | Functions |
| 7 | Arrays |
| 8 | Pointers: The Core Concept |
| 9 | Pointers and Arrays Together |
| 10 | Strings |
| 11 | Structures, Unions and Enums |
| 12 | Dynamic Memory: malloc and free |
| 13 | File Input and Output |
| 14 | The Preprocessor |
| 15 | Multi-File Programs and Linkage |
| 16 | The Standard Library Tour |

PART II **EMBEDDED AND SYSTEMS C**

- | | |
|----|--|
| 17 | Bit Manipulation |
| 18 | Fixed-Point Arithmetic |
| 19 | Memory Layout: Flash, SRAM, Stack, Heap |
| 20 | volatile, const and restrict |
| 21 | Programming Without malloc |
| 22 | Function Pointers and Dispatch Tables |
| 23 | Alignment, Padding and Endianness |
| 24 | Integer Overflow and Undefined Behaviour |
| 25 | Floating Point on Microcontrollers |
| 26 | Binary File Formats and Serialisation |
| 27 | ARM Cortex-M Specifics |
| 28 | Debugging, Testing and Optimisation |

A **Exercise Solutions**

B **Quick Reference**

PART I

General-Purpose C

The language itself: syntax, types, memory, pointers and the standard library. Everything here works the same on a laptop as on a microcontroller.

Chapter 1

Getting Started

Your first program, the compiler, and what actually happens

1.1 What C Is, and Why It Still Matters

C was written in 1972 to build an operating system. That origin shapes everything about it. C gives you direct access to memory addresses, exact control over how many bytes a value occupies, and almost no runtime machinery between your code and the processor. There is no garbage collector, no bounds checking, no exception handling. What you write is close to what the CPU executes.

This is a liability on a desktop, where those safety features cost little and prevent whole classes of bugs. It is an asset on a microcontroller, where there may be 320 KB of RAM in total and no operating system at all. The ODT library that supplies examples throughout this book trains a neural network on a chip with less memory than a single web page uses — that is only possible in a language where you control every byte.

Where C is used today	
Operating system kernels	Linux, Windows NT core, macOS XNU
Embedded firmware	Microcontrollers, automotive ECUs, medical devices
Language runtimes	CPython, Ruby MRI, Lua, PHP are written in C
Databases	SQLite, PostgreSQL, Redis
Performance libraries	zlib, OpenSSL, FFmpeg, BLAS kernels

1.2 The Smallest Complete Program

hello.c

```
#include <stdio.h>

int main(void) {
    printf("Hello, world\n");
    return 0;
}
```

Every line, explained

#include <stdio.h> — This is a preprocessor directive, not a C statement. Before the compiler proper runs, the preprocessor finds the file `stdio.h` and pastes its entire contents in at this point. That file contains the *declaration* of `printf`: it tells the compiler that such a function exists, what arguments it takes, and what it returns. Without this line the compiler would not know what `printf` means.

The angle brackets mean "look in the system include directories". Double quotes — `#include "myheader.h"` — mean "look in my project directory first". Use angle brackets for standard headers, quotes for your own.

int main(void) — Execution begins here. Every C program has exactly one `main`. The `int` before it says `main` returns an integer to the operating system. The `(void)` says it takes no arguments.

int main() is not the same as int main(void)

In C (unlike C++), empty parentheses mean "unspecified arguments" — the compiler will not check calls against the definition. Always write (void) explicitly when a function takes nothing. This applies to every function, not just main.

printf("Hello, world\\n"); — A function call. The text in double quotes is a *string literal*. The \\n inside it is an escape sequence for the newline character — a single byte with value 10, not two characters. The semicolon ends the statement; C requires one after every statement.

return 0; — Hands the value 0 back to whatever launched the program. By universal convention 0 means success and any non-zero value means failure. A shell script can check this.

{ } — Braces group statements into a *block*. The function body is a block. Blocks also create scope, which Chapter 2 covers.

1.3 Compiling: What Actually Happens

Turning hello.c into a running program takes four distinct stages. Most of the time a single command runs all four, but understanding them separately makes error messages far easier to read.

Stage	
1. Preprocess	Expands #include, #define, #if. Output is still C text, just longer. See it with gcc -E hello.c
2. Compile	Translates C into assembly for your CPU. Syntax errors and type errors appear here. See it with gcc -S hello.c
3. Assemble	Turns assembly into machine code in an object file (.o). gcc -c hello.c
4. Link	Combines object files and libraries into one executable. "Undefined reference" errors happen here, not at compile time.

Building and running

```
# All four stages at once
gcc -Wall -Wextra -std=c11 hello.c -o hello
./hello

# Check the exit code that return 0 produced
echo $?          # prints 0
```

Always compile with warnings on

-Wall -Wextra turn on warnings that catch real bugs: unused variables, missing return values, comparisons between signed and unsigned, uninitialised reads. C will happily compile code that is badly broken. The warnings are the only thing standing between you and a silent failure. Treat every warning as an error you have not fixed yet.

Useful compiler flags	
-Wall -Wextra	Essential warnings. Use always.
-std=c11	Use the C11 standard rather than the compiler default
-g	Include debug symbols so gdb can show variable names
-O0	No optimisation. Use while debugging.

Useful compiler flags	
-O2	Standard optimisation for release builds
-Os	Optimise for code size. What embedded builds use.
-fsanitize=address	Catch buffer overruns and use-after-free at runtime

1.4 Reading Compiler Errors

C compiler messages look intimidating but follow a rigid format: file name, line number, column, severity, message. The first error is the one to fix — later errors are frequently cascading nonsense caused by the first.

A typical error

```
hello.c:4:5: error: expected ';' before 'return'
  4 |     printf("Hello")
    |                               ^
    |                               ;
  5 |     return 0;
    |     ~~~~~
```

This says: in `hello.c`, line 4, column 5, a semicolon was expected before the word `return`. GCC even shows where to insert it. The missing semicolon is on line 4, but the compiler only noticed when it reached line 5 — this offset by one line is extremely common.

1.5 A Slightly Larger Program

Here is a program with variables, input, arithmetic, and formatted output. Everything in it is explained fully in the next two chapters; read it now for shape, not detail.

har_window.c

```
#include <stdio.h>

int main(void) {
    int    samples    = 128;        /* one HAR window is 128 readings */
    int    channels    = 9;         /* 3 accel + 3 gyro + 3 body accel */
    float  sampleRate  = 50.0f;     /* Hz */

    int    totalValues = samples * channels;
    float  windowSecs  = samples / sampleRate;

    printf("Window length : %d samples\n", samples);
    printf("Channels      : %d\n", channels);
    printf("Total values   : %d\n", totalValues);
    printf("Duration      : %.2f seconds\n", windowSecs);
    printf("Bytes (float)  : %zu\n", totalValues * sizeof(float));

    return 0;
}
```

Output

```
Window length : 128 samples
Channels      : 9
Total values   : 1152
Duration      : 2.56 seconds
Bytes (float) : 4608
```

This is real HAR geometry

Those numbers are the actual shape of one input to the ODT network. The UCI HAR dataset samples a phone IMU at 50 Hz and cuts the stream into 2.56-second windows of 128 readings. Each reading has 9 channels. One window is therefore 1152 float values — 4608 bytes. With 320 KB of SRAM on the board, you can hold roughly 70 such windows in memory at once, and that is before the model weights, the gradients, and the optimiser state. Memory arithmetic like this is a daily activity in embedded C.

1.6 Comments

```
/* Block comment.  
   Spans multiple lines.  
   Cannot be nested - /* this breaks */ and the rest is a syntax error */  
  
// Line comment. Runs to end of line. Standard since C99.  
  
int x = 5; // trailing comment
```

Write comments that explain *why*, not *what*. `i++; // increment i is noise.` `i++; // skip the header byte is information.`

Exercises — Chapter 1

1. Write and compile a program that prints your name and the current year on two separate lines.
2. Deliberately remove a semicolon from a working program. Compile it. Read the error message carefully and note which line number it reports versus which line the error is really on.
3. Modify `har_window.c` to also print the total bytes needed for a batch of 4 windows. What is the answer?
4. Run `gcc -E hello.c | wc -l`. How many lines does the preprocessor produce from a 6-line file? Why so many?
5. A program returns 3 from `main`. Write the shell command that shows this value after running it.

Chapter 2

Variables, Types and Memory

What a variable really is, and how many bytes it costs

2.1 A Variable Is a Named Box of Bytes

When you write `int x = 5;` three things happen. The compiler reserves some bytes of memory — four, on almost every modern machine. It records that the name `x` refers to those bytes. And it arranges for the bit pattern representing 5 to be written there.

The *type* is not stored in memory. It exists only at compile time, and it tells the compiler two things: how many bytes to reserve, and how to interpret the bits in them. The same four bytes could be an `int`, a `float`, or part of a pointer — the bits do not know. Only the type tells the compiler how to read them.

The same bits, two interpretations

```
int    a = 1078530011;
float  b;

/* Copy the raw bytes of a into b without converting */
memcpy(&b, &a, sizeof(float));

printf("%d\n", a);    /* 1078530011 */
printf("%f\n", b);    /* 3.141593 - same bits, different type */
```

2.2 The Integer Types

C has an unfortunate history here. The original types — `short`, `int`, `long` — have sizes that vary between platforms. The standard only guarantees minimums and an ordering. On your laptop `int` is 32 bits; on some embedded compilers it is 16.

Type	Typical bytes	Guarantee
<code>char</code>	1	At least 8 bits. Signedness is implementation-defined.
<code>short</code>	2	At least 16 bits
<code>int</code>	4	At least 16 bits. Usually the natural word size.
<code>long</code>	4 or 8	At least 32 bits. 8 on 64-bit Linux, 4 on Windows.
<code>long long</code>	8	At least 64 bits. C99 onward.

Never rely on these sizes

Code that assumes `int` is 32 bits will silently break when moved to a 16-bit microcontroller. A loop counter that overflows at 32767 instead of 2147483647 produces wrong results with no error message. This is one of the most common portability failures in C.

2.3 Fixed-Width Types: What to Use Instead

C99 added `<stdint.h>`, which defines types with guaranteed exact widths. In embedded code you should use these almost exclusively.

Type	Bytes	Range
<code>int8_t / uint8_t</code>	1	-128..127 / 0..255
<code>int16_t / uint16_t</code>	2	-32768..32767 / 0..65535
<code>int32_t / uint32_t</code>	4	-2147483648..2147483647 / 0..4294967295
<code>int64_t / uint64_t</code>	8	about +/- 9.2×10^{18} / $0..1.8 \times 10^{19}$
<code>size_t</code>	4 or 8	Unsigned. Big enough for any object size. Use for indices and byte counts.
<code>ptrdiff_t</code>	4 or 8	Signed. Result of subtracting two pointers.

ODT uses these everywhere

Look at any ODT signature and you will see fixed-width types: `bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex)`. `size_t` for the index because it is a count that can never be negative and must be able to address any element. Inside, the tensor data is `uint8_t *` — explicitly one byte per element, because the code does bit arithmetic on it and needs to know exactly how wide each unit is. If that were unsigned `char *` the code would still work, but `uint8_t` states the intent.

Preferred style

```
#include <stdint.h>
#include <stddef.h>

uint8_t maskByte = 0xFF;      /* exactly 8 bits, no doubt */
int32_t accumulator = 0;      /* exactly 32 bits for int8 x int8 sums */
size_t elementCount = 1152;   /* an index or count, never negative */
```

2.4 Signed and Unsigned

An unsigned type uses all its bits for magnitude. A signed type reserves the top bit as a sign bit and stores negative values in *two's complement*: to negate a number, invert every bit and add one.

Two's complement

```
5 as int8_t = 0b00000101
-5 as int8_t = 0b11111011

Derivation: invert 0b00000101 -> 0b11111010
            add 1          -> 0b11111011
```

Two's complement has a useful property: the same addition circuit works for signed and unsigned values. $0b11111011 + 0b00000101 = 0b00000000$ with a carry out, which is correct for $-5 + 5 = 0$.

Mixing signed and unsigned is a trap

When you compare a signed value with an unsigned one, C converts the signed value to unsigned first. `-1 < 1u` evaluates to **false**, because `-1` becomes `4294967295`. This bites hardest in loops: `for (size_t i = n - 1; i >= 0; i--)` never terminates, because a `size_t` is always ≥ 0 . When it wraps below zero it becomes a huge positive number.

Countdown loops with unsigned

```
/* WRONG - infinite loop */
for (size_t i = n - 1; i >= 0; i--) { ... }

/* Correct - test before decrementing */
for (size_t i = n; i > 0; i--) {
    size_t idx = i - 1;    /* real index */
    ...
}

/* Also correct - what ODT uses in Fisher-Yates shuffle */
for (size_t i = n - 1; i > 0; i--) {
    size_t j = rngBounded(i + 1);
    swap(&arr[i], &arr[j]);
}
/* index 0 needs no swap, so stopping at i > 0 is correct here */
```

2.5 Floating-Point Types

Type	Bytes	Mantissa	Precision
float	4	23+1 bits mantissa	about 7 decimal digits
double	8	52+1 bits mantissa	about 15 decimal digits
long double	10-16	platform-specific	rarely used

A float stores a number as sign, exponent and mantissa: $\text{value} = \text{sign} \times 1.\text{mantissa} \times 2^{\text{exponent}}$. This means the spacing between representable numbers grows with magnitude. Near 1.0 the gap is about $1.2\text{e-}7$. Near 1000000.0 it is about 0.06.

0.1 + 0.2 is not 0.3

Neither 0.1 nor 0.2 is exactly representable in binary floating point, any more than $1/3$ is exactly representable in decimal. The sum is 0.30000000000000004. Never compare floats with `==`. Compare against a tolerance: `fabsf(a - b) < 1e-6f`.

Why ODT stores AdamW betas as double

The AdamW optimiser uses $\text{beta2} = 0.999$ and needs the value $1 - \text{beta2} = 0.001$. In float32, `1.0f - 0.999f` gives 0.00100004673 — four correct digits out of seven, because subtracting two nearly-equal numbers destroys precision (this is called *catastrophic cancellation*). AdamW then raises beta2 to the power of the step count, thousands of times, and the error compounds. ODT declares `beta1`, `beta2`, `eps` and `weightDecay` as `double` and casts to float only at the very last moment, when writing into the weight tensor. Everything else in the library is float32; these four scalars are the deliberate exception.

2.6 Declaration, Initialisation, and the Uninitialised Trap

```
int a;          /* declared, NOT initialised - contains garbage */
int b = 0;      /* declared and initialised */
int c = b + 1;  /* initialiser can be any expression */

static int d;   /* static/global: guaranteed zero-initialised */
```

Reading an uninitialised local is undefined behaviour

A local variable with no initialiser contains whatever bytes happened to be on the stack. It is not zero, and it is not even guaranteed to be the same value twice. Reading it is undefined behaviour — the compiler is permitted to assume it never happens and optimise accordingly, producing results that make no sense. Always initialise. `-Wall` catches most cases.

Globals and static variables are different: the C standard requires them to be zero-initialised before `main` runs. On an embedded target this is done by startup code that zeroes the `.bss` section — Chapter 19 covers exactly how.

2.7 Scope and Lifetime

Scope is where a name is visible. *Lifetime* is how long the storage exists. They are related but not the same.

```
int global = 1;           /* file scope, whole-program lifetime */

void f(void) {
    int local = 2;        /* block scope, lifetime = this call */
    static int counter = 0; /* block scope, whole-program lifetime */
    counter++;            /* remembers its value between calls */

    {
        int inner = 3;    /* visible only inside these braces */
        int local = 99;    /* SHADOWS the outer local - legal, confusing */
        printf("%d\n", local); /* prints 99 */
    }
    printf("%d\n", local); /* prints 2 - inner one is gone */
}
```

Declaration	Scope	Lifetime	Where stored
(none), inside a function	Block	Until the block exits	Stack
static, inside a function	Block	Whole program	.data / .bss
(none), at file level	Whole program	Whole program	.data / .bss
static, at file level	This file only	Whole program	.data / .bss

static at file level means private

Marking a file-level function or variable `static` hides it from other translation units. This is C's only encapsulation mechanism. ODT uses it for the RNG state: `static uint32_t state;` in `RNG.c` means no other file can touch it directly — they must go through `rngSeed()` and `rngNext()`.

2.8 Constants and Literals

Literal forms

```

42      /* int */
42u     /* unsigned int */
42L     /* long */
42ULL   /* unsigned long long */

3.14    /* double - note: NOT float */
3.14f   /* float */
0x1p24f /* hex float: 1 x 2^24 = 16777216.0f, exact */

0x2A    /* hexadecimal 42 */
0b101010 /* binary 42 - C23 standard, GCC extension before that */
052     /* OCTAL 42 - a leading zero means base 8! */

'A'     /* int with value 65, not a char */
"text"  /* array of 5 chars: t,e,x,t,\0 */

```

A leading zero means octal

Writing `int port = 0755;` does not give 755. It gives 493, because 0755 is octal. This bites people writing zero-padded numbers such as dates or IP octets.

0x1p24f in the ODT RNG

ODT converts a random 32-bit integer to a float in $[0,1)$ with `(rngNext() >> 8) / 0x1p24f`. The literal `0x1p24f` is hexadecimal floating-point notation meaning 1×2^{24} , exactly 16777216.0f with no rounding whatsoever. Writing 16777216.0f would work too, but the hex-float form makes the power-of-two intent unmistakable, and guarantees the value is exact even if someone later changes the shift amount.

2.9 sizeof

`sizeof` is an operator, not a function, and it is evaluated at compile time. It yields the number of bytes a type or object occupies, as a `size_t`.

```

sizeof(int)      /* 4 on most platforms */
sizeof(float)    /* 4 */
sizeof(double)   /* 8 */
sizeof x         /* parentheses optional for an object */

int arr[10];
sizeof(arr)      /* 40 - the whole array */
sizeof(arr) / sizeof(arr[0]) /* 10 - element count idiom */

/* Print a size_t with %zu, not %d */
printf("%zu\n", sizeof(float));

```

sizeof on a pointer gives the pointer size

Inside a function that receives an array as a parameter, the array has already decayed to a pointer. `sizeof(param)` gives 4 or 8, not the array size. This is why C functions taking arrays must also take a length. Chapter 9 explains decay in full.

Exercises — Chapter 2

1. Print `sizeof` for `char`, `short`, `int`, `long`, `long long`, `float`, `double` and `size_t` on your machine. Do any surprise you?
2. Write a program that stores 200 in an `int8_t` and prints it. Explain the output.
3. Predict the output of `printf("%d\n", -1 < 1u);` then run it. Explain.
4. Write a loop that counts down from `n-1` to 0 using a `size_t`, without an infinite loop.
5. Compute in float32: `1.0f - 0.999f`. Compute the same in double. Print both with `%.10f`. How many digits agree?
6. A static local counter is incremented once per call. Write a function that returns how many times it has been called.
7. What does `int x = 010 + 10;` assign? Why?

Chapter 3

Operators and Expressions

Arithmetic, comparison, logic, and the rules that surprise people

3.1 Arithmetic Operators

Operator	
+ - * /	Add, subtract, multiply, divide
%	Remainder. Integers only - not defined for floats.
++ --	Increment / decrement by one
-X	Unary negation

Integer division truncates

$7 / 2$ is 3, not 3.5. Both operands are int, so C does integer division and discards the remainder. To get 3.5 at least one operand must be floating point: $7.0 / 2$ or $(\text{float})7 / 2$. This is the single most common arithmetic bug in beginner C.

```
int    a = 7 / 2;           /* 3 - integer division */
float  b = 7 / 2;           /* 3.0 - divided as ints FIRST, then converted */
float  c = 7.0f / 2;        /* 3.5 - correct */
float  d = (float)7 / 2;     /* 3.5 - cast forces float division */
```

Note the trap in line 2. The division happens before the assignment, using integer rules, and only the already-truncated result is converted to float. The cast must be applied to an operand, not to the result.

The truncating-division idiom

Integer truncation is useful when deliberate. Two idioms appear constantly:

```
/* Which byte holds bit i? (floor division) */
size_t byteIndex = i / 8;

/* Which bit within that byte? */
size_t bitIndex  = i % 8;

/* Ceiling division: how many bytes hold n bits? */
size_t byteCount = (n + 7) / 8;
```

Exactly what tensorBooGet does

ODT packs one boolean per bit. $\text{flatIndex} / 8$ selects the byte and $\text{flatIndex} \% 8$ selects the bit inside it. For a sparse mask over 524288 weights, $(524288 + 7) / 8 = 65536$ bytes — 64 KB instead of 512 KB. That ceiling-division idiom appears in `calcBytesPerTensor` and is why a `RigL` mask fits in SRAM at all.

3.2 Comparison and Logical Operators

Operator	
== !=	Equal, not equal
< > <= >=	Ordering
&&	Logical AND. Short-circuits.
	Logical OR. Short-circuits.
!	Logical NOT

C has no boolean type historically. Comparisons yield `int`: 1 for true, 0 for false. In a condition, **any** non-zero value is true. C99 added `<stdbool.h>` with `bool`, `true` and `false`, which are worth using for clarity.

Short-circuit evaluation

`&&` and `||` stop as soon as the answer is known. If the left side of `&&` is false, the right side is **never evaluated**. This is not an optimisation; it is guaranteed behaviour you can rely on.

Short-circuit as a guard

```
/* Safe: if p is NULL, p->field is never touched */
if (p != NULL && p->field > 0) { ... }

/* Safe: bounds checked before the array access */
if (i < n && arr[i] == target) { ... }

/* Reversing the order is a crash waiting to happen */
if (arr[i] == target && i < n) { ... } /* WRONG */
```

= is not ==

`if (x = 5)` assigns 5 to `x` and then tests 5, which is truthy — so the branch always runs. This compiles without error. `-Wall` warns about it. Some people write `if (5 == x)` so a typo becomes a compile error, though modern warnings make this less necessary.

3.3 Bitwise Operators

These work on individual bits. They are the foundation of Chapter 17 and of everything ODT does with masks, so they deserve care here.

Op	Name	Effect	Typical use
&	AND	Bit is 1 only if both inputs are 1	Clear bits / test bits
	OR	Bit is 1 if either input is 1	Set bits
^	XOR	Bit is 1 if inputs differ	Toggle bits / RNG
~	NOT	Inverts every bit	Build inverse masks
<<	Left shift	Shift left, fill with 0	Multiply by 2^n
>>	Right shift	Shift right	Divide by 2^n

Worked bitwise examples


```

uint8_t a = 0b10110100; /* 180 */
uint8_t b = 0b00111000; /* 56 */

a & b -> 0b00110000 /* 48 - bits set in BOTH */
a | b -> 0b10111100 /* 188 - bits set in EITHER */
a ^ b -> 0b10001100 /* 140 - bits that DIFFER */
~a -> 0b01001011 /* 75 - every bit flipped */
a << 1 -> 0b01101000 /* top bit shifted out and lost */
a >> 2 -> 0b00101101 /* bottom two bits lost */

```

The three fundamental bit idioms

Memorise these four

```

/* TEST bit n: isolate it and compare */
bool isSet = (value >> n) & 1u;
/* or equivalently */
bool isSet = (value & (1u << n)) != 0;

/* SET bit n to 1: OR with a mask that has only bit n */
value |= (1u << n);

/* CLEAR bit n to 0: AND with a mask that has every bit EXCEPT n */
value &= ~(1u << n);

/* TOGGLE bit n: XOR with a mask that has only bit n */
value ^= (1u << n);

```

tensorBoolGet and tensorBoolSet, decoded

ODT uses exactly the first three idioms. `return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;` is the TEST idiom. `data[byteIndex] |= (uint8_t)(1u << bitIndex);` is SET. `data[byteIndex] &= (uint8_t)~(1u << bitIndex);` is CLEAR. The `(uint8_t)` casts exist because `1u << bitIndex` is a 32-bit unsigned int; casting back to 8 bits makes the narrowing explicit and silences the warning.

Right-shifting a negative number is implementation-defined

For signed types, `>>` may fill the top bits with the sign bit (arithmetic shift, what every real compiler does) or with zeros (logical shift). The standard permits either. Always shift unsigned types. Shifting by more than the width of the type, or by a negative amount, is outright undefined behaviour.

3.4 Assignment and Compound Assignment

```

x = 5; /* plain assignment */
x += 3; /* same as x = x + 3 */
x -= 3; x *= 3; x /= 3; x %= 3;
x &= mask; x |= mask; x ^= mask;
x <<= 2; x >>= 2;

```

Compound assignment evaluates the left operand exactly once. This matters when it has side effects: `arr[f()] += 1;` calls `f` once, whereas `arr[f()] = arr[f()] + 1;` calls it twice.

Assignment is an *expression* in C, and its value is the value assigned. That is why `a = b = c = 0;` works, and why `if (x = 5)` compiles.

3.5 Increment and Decrement

```
int i = 5;
int a = i++; /* POST: a gets 5, then i becomes 6 */

int j = 5;
int b = ++j; /* PRE: j becomes 6, then b gets 6 */
```

When the value is discarded — as in a for loop header — the two forms are identical and compilers generate the same code. Use whichever reads better; `i++` is conventional.

Never modify a variable twice in one expression

`i = i++ + 1;` and `arr[i] = i++;` are undefined behaviour. C does not specify the order in which subexpressions are evaluated, so the compiler may produce anything at all — including different results at different optimisation levels. Split such code into separate statements.

3.6 The Conditional (Ternary) Operator

```
int max = (a > b) ? a : b;

/* Clamping - very common in quantisation code */
int8_t q = (r < -128) ? -128 : (r > 127) ? 127 : (int8_t)r;

/* ODT: guard against divide-by-zero in the RigL cosine schedule */
float progress = (float)step / (float)(totalSteps > 0 ? totalSteps : 1);
```

The ternary is an expression, so it can appear where a statement cannot — inside an initialiser, a function argument, or a return. Keep it to one level of nesting; beyond that an `if` is clearer.

3.7 Operator Precedence

C has fifteen precedence levels. Rather than memorise all of them, learn the groups that actually cause bugs and parenthesise everything else.

	Operators	Notes
Highest	<code>() [] -> .</code>	Call, subscript, member
	<code>! ~ ++ -- (cast) * & sizeof</code>	Unary (right-to-left)
	<code>* / %</code>	Multiplicative
	<code>+ -</code>	Additive
	<code><< >></code>	Shifts — lower than + and -
	<code>< <= > >=</code>	Relational
	<code>== !=</code>	Equality
	<code>&</code>	Bitwise AND — lower than ==
	<code>^</code>	Bitwise XOR
	<code> </code>	Bitwise OR
	<code>&&</code>	Logical AND

	Operators	Notes
		Logical OR
	?:	Conditional (right-to-left)
Lowest	= += -= ...	Assignment (right-to-left)

The two precedence bugs everyone hits

Shifts bind looser than arithmetic. $1 \ll n + 1$ is $1 \ll (n+1)$, not $(1 \ll n) + 1$.

Bitwise AND binds looser than equality. $x \& 1 == 0$ is $x \& (1 == 0)$, which is $x \& 0$, which is always 0. You wanted $(x \& 1) == 0$. This dates back to C predating $\&\&$, and it can never be fixed without breaking old code.

3.8 Type Conversion

Implicit: the usual arithmetic conversions

When operands of an operator have different types, C converts them to a common type before operating. Roughly: anything narrower than `int` is promoted to `int`; then the narrower of the two is converted to the wider; and if one is unsigned and the same rank, the signed one becomes unsigned.

```
uint8_t a = 200, b = 100;
uint8_t c = a + b;      /* a,b promoted to int -> 300 -> truncated to 44 */
int      d = a + b;      /* 300 - no truncation, int is wide enough */

int8_t e = -1;
if (e < 1u) { }          /* FALSE - e converts to 4294967295 */
```

Explicit: casts

```
float f = (float)intValue; /* value-preserving conversion */
int i = (int)3.99f;        /* truncates toward zero -> 3 */
int j = (int)-3.99f;       /* -3, not -4 */
uint8_t small = (uint8_t)largeInt; /* keeps only the low 8 bits */
```

A cast means "I know what I am doing"

Casts silence warnings. That is their purpose and their danger. Every cast is a place where you have told the compiler to stop checking. Before writing one, ask whether the types should have matched in the first place. Legitimate uses: converting `int` to `float` for division, narrowing after a deliberate mask, and converting `void *` from `malloc`.

Cast discipline in ODT quantisation

The float-to-int8 conversion in ODT reads: `int32_t r = roundSRHalfAway(f[i] * inv);` then `r = r < -128 ? -128 : (r > 127 ? 127 : r);` then `d[i] = (int8_t)r;`. Three separate steps: round in float, clamp in int32, narrow to int8. The clamp is what makes the final cast safe. Casting an out-of-range float directly to `int8_t` is undefined behaviour, and on ARM it silently produces garbage rather than crashing.

Exercises — Chapter 3

1. Without running it, predict `printf("%d", 17 / 5 * 5 + 17 % 5);`. Then run it. Explain why the answer is what it is.
2. Write an expression that tests whether bit 6 of a `uint8_t` is set, using each of the two TEST idioms.
3. Write a function `uint8_t toggleBit(uint8_t v, int n)`.
4. Explain why `x & 1 == 0` is always false and fix it.
5. What is the value of `1 << 3 + 1`? Add parentheses to make it 9 instead.
6. Given `uint8_t a = 200, b = 100;`, explain why `(uint8_t)(a+b)` is 44.
7. Write a one-line clamp that limits an `int32_t` to the range `[0, 255]` using the ternary operator.
8. Implement `getBitmask(start, end)` from the ODT codebase yourself, without looking, then compare.

Chapter 4

Control Flow: Branching

if, else, switch, and how to structure decisions

4.1 if and else

```
if (condition) {
    /* runs when condition is non-zero */
} else if (other) {
    /* runs when condition was zero and other is non-zero */
} else {
    /* runs when neither matched */
}
```

The condition is any expression. C tests it against zero: zero is false, everything else is true. There is no requirement that it be a comparison.

```
if (n)          { } /* true when n != 0 */
if (ptr)        { } /* true when ptr is not NULL */
if (!ptr)       { } /* true when ptr IS NULL */
if (strcmp(a,b) == 0) { } /* strings equal - strcmp returns 0 on match */
```

Always use braces

C allows a single statement without braces: `if (x) doThing();`. This caused the Apple "goto fail" TLS vulnerability in 2014, where a duplicated line ended up outside an unbraced if and skipped certificate validation entirely. Always write the braces, even for one line. The cost is two characters; the benefit is that adding a second statement later cannot silently break the logic.

The dangling else

```
/* An else binds to the NEAREST unmatched if */
if (a)
    if (b)
        x();
else
    y();          /* binds to if(b), NOT if(a) - despite the indentation */
```

Braces remove the ambiguity entirely. This is also why the `do { ... } while(false)` macro idiom exists — Chapter 14 covers it.

4.2 switch

```

switch (expression) {
    case 1:
        handleOne();
        break;
    case 2:
    case 3: /* deliberate fall-through: 2 and 3 share code */
        handleTwoOrThree();
        break;
    default:
        handleOther();
        break;
}

```

The controlling expression must be an integer type — int, char, enum. It cannot be a float, a string, or a pointer. Each case label must be a compile-time constant, and duplicates are a compile error.

Fall-through is the default

Without `break`, execution continues into the next case. Sometimes this is what you want (cases 2 and 3 above). Far more often it is a bug. Compile with `-Wimplicit-fallthrough` and write an explicit comment `/* fall through */` when you mean it.

switch on an enum, and omit default

If you switch on an enum and cover every enumerator, leave out `default`. Then when someone adds a new enumerator later, `-Wswitch` warns you about the switch that no longer handles every case. Adding a `default` silences that warning and loses the safety net.

Where ODT uses switch, and where it does not

ODT dispatches quantisation conversion with a switch on `qtype_t`, because the set of types is fixed and small. But it dispatches *layers* with a function-pointer table, not a switch: `layerFunctions[LINEAR] = {linearForward, linearBackward, ...}`. The reason is extensibility — adding a layer type means adding one table row rather than editing three switch statements scattered across the codebase. Chapter 22 covers this pattern in depth.

4.3 Structuring Decisions Well

Early return over deep nesting

```

/* Hard to follow */
int process(data_t *d) {
    if (d != NULL) {
        if (d->size > 0) {
            if (d->buffer != NULL) {
                return doWork(d);
            } else { return -3; }
        } else { return -2; }
    } else { return -1; }
}

/* Same logic, flat - "guard clause" style */
int process(data_t *d) {
    if (d == NULL) return -1;
    if (d->size <= 0) return -2;
    if (d->buffer == NULL) return -3;
    return doWork(d);
}

```

The second version puts the error cases first and short, leaving the happy path unindented at the bottom. Every ODT function that validates a tensor uses this shape.

Exercises — Chapter 4

1. Write a function classifying an int as negative, zero, or positive, using if/else if/else.
2. Rewrite it using nested ternaries. Which is more readable?
3. Write a switch that maps the HAR class indices 0-5 to their activity names (walking, walking upstairs, walking downstairs, sitting, standing, laying).
4. Take a deeply nested validation function and rewrite it in guard-clause style.
5. Explain what happens if you remove all the break statements from a five-case switch and the expression matches case 1.

Chapter 5

Control Flow: Loops

for, while, do-while, and iterating over data correctly

5.1 The Three Loop Forms

```
/* for: when you know the iteration pattern up front */
for (init; condition; update) {
    body
}

/* while: when the condition is checked before each pass */
while (condition) {
    body
}

/* do-while: body runs at least once, condition checked after */
do {
    body
} while (condition);
```

A for loop is exactly equivalent to a while loop with the init before it and the update at the end of the body — except that `continue` still runs the update in a for, but skips it in the rewritten while. That difference matters.

5.2 Iterating Over Data

```
/* The canonical form. Note: i < n, not i <= n */
for (size_t i = 0; i < n; i++) {
    process(arr[i]);
}
```

Off-by-one: the classic C bug

An array of n elements has valid indices 0 through $n-1$. Writing `i <= n` reads one element past the end. C does not check this. It may work by accident, corrupt an adjacent variable, or crash — and which one happens can change between builds.

Flat iteration over a tensor

Nearly every ODT kernel is a flat loop over the tensor:

```
size_t n = calcNumberOfElementsByTensor(weights);
for (size_t i = 0; i < n; i++) {
    if (!tensorBoolGet(mask, i)) continue;
    w[i] -= lr * g[i];
}
```

The loop variable `i` is the flat index that `tensorBoolGet` expects. A 2D weight matrix of shape `[512, 1024]` has 524288 elements, and this single loop visits all of them in memory order — which is also the cache-friendly order on the Cortex-M7.

5.3 Nested Loops and Multi-Dimensional Data

```
/* HAR input tensor: [batch][channel][timestep] */
for (size_t b = 0; b < batch; b++) {
    for (size_t c = 0; c < channels; c++) {          /* 9 */
        for (size_t t = 0; t < timesteps; t++) {    /* 128 */
            size_t flat = b*channels*timesteps + c*timesteps + t;
            float value = data[flat];
            ...
        }
    }
}
```

The innermost loop varies the last index. That is deliberate: in row-major layout, consecutive values of *t* are consecutive in memory. Reversing the loop order would still be correct but would jump across memory on every iteration, destroying cache performance. On the STM32F746, with a 4 KB data cache, this can be a several-fold slowdown.

Loop order follows memory order

Whenever you nest loops over a multi-dimensional array, make the innermost loop iterate over the **last** index. This is the single easiest performance win in numeric C.

5.4 break and continue

```
for (size_t i = 0; i < n; i++) {
    if (arr[i] == 0) continue; /* skip to next iteration */
    if (arr[i] < 0) break;     /* leave the loop entirely */
    process(arr[i]);
}
```

`break` exits only the *innermost* enclosing loop or switch. To leave several levels at once, either use a flag, extract the loops into a function and `return`, or — rarely, and only for error cleanup — use `goto`.

```
/* Escaping nested loops with a function */
static bool findValue(int **grid, size_t rows, size_t cols, int target,
                     size_t *outR, size_t *outC) {
    for (size_t r = 0; r < rows; r++)
        for (size_t c = 0; c < cols; c++)
            if (grid[r][c] == target) {
                *outR = r; *outC = c;
                return true; /* leaves BOTH loops */
            }
    return false;
}
```

5.5 do-while and the Rejection-Sampling Pattern

`do-while` is rare, but it is exactly right when the body must run before the condition can be evaluated. Rejection sampling is the textbook case.

```

/* ODT RNG.c - unbiased bounded random number */
uint32_t rngBounded(uint32_t bound) {
    uint32_t limit = UINT32_MAX - (UINT32_MAX % bound);
    uint32_t x;
    do {
        x = rngNext();          /* must draw before we can test */
    } while (x >= limit);      /* redraw if in the biased tail */
    return x % bound;
}

```

Why the rejection loop is there

A naive `rngNext() % bound` is biased. If `UINT32_MAX` is not an exact multiple of `bound`, the low residues occur one extra time each. For `bound=3`, values 0 and 1 appear 1431655766 times but 2 appears only 1431655765 — a tiny bias, but Fisher-Yates shuffling relies on perfect uniformity to guarantee that every permutation is equally likely. The rejection loop discards the unbalanced tail. It rejects at most half the values in the worst case, so the expected number of iterations is under 2.

5.6 Infinite Loops

```

for (;;) { ... }    /* idiomatic C */
while (1) { ... }   /* equally common */

```

These are not a mistake in embedded code — they are the norm. A microcontroller program never exits; `main` ends in an infinite loop that services events forever. If `main` returned on an STM32, the startup code would jump into undefined memory.

Typical embedded main

```

int main(void) {
    hardwareInit();
    modelInit();

    for (;;) {                /* never returns */
        if (windowReady()) {
            trainOneStep();
        }
        __WFI();              /* sleep until next interrupt */
    }
}

```

Exercises — Chapter 5

1. Write a loop summing an array of 1152 floats (one HAR window). Print the mean.
2. Write the same loop three ways: for, while, and do-while.
3. Given a [4][9][128] tensor stored row-major, write the flat-index expression for element [2][5][100]. Compute the number.
4. Write nested loops that print a 9x9 multiplication table.
5. Explain why `for (size_t i = n; i >= 0; i--)` never terminates, and write a correct countdown.
6. Implement Fisher-Yates shuffle for an `int` array using `rand() % (i+1)`. Then explain what bias that introduces.
7. Write a loop that finds the largest absolute value in a float array, skipping any element whose mask bit is zero.

Functions

Decomposition, parameters, return values, and the call stack

6.1 Definition and Declaration

```
/* Declaration (prototype) - tells the compiler the signature.
   Usually lives in a .h file. Note the semicolon. */
float meanOf(const float *data, size_t n);

/* Definition - the actual code. Lives in a .c file. */
float meanOf(const float *data, size_t n) {
    if (n == 0) return 0.0f;
    float sum = 0.0f;
    for (size_t i = 0; i < n; i++) {
        sum += data[i];
    }
    return sum / (float)n;
}
```

The compiler processes a file top to bottom. If you call a function before it has seen a declaration, older C would guess the signature (assuming it returns `int`) and produce wrong code silently. C99 made this an error. Declare before use, which in practice means putting prototypes in a header.

6.2 Everything Is Passed By Value

C has exactly one parameter-passing mechanism: the value of the argument is copied into the parameter. Modifying the parameter inside the function has no effect on the caller's variable.

```
void tryToModify(int x) {
    x = 99;          /* modifies the local copy only */
}

int a = 5;
tryToModify(a);
printf("%d\n", a);  /* still 5 */
```

To modify the caller's variable, pass its address and modify through the pointer. The pointer itself is still passed by value — but the thing it points at is shared.

```
void actuallyModify(int *x) {
    *x = 99;         /* writes through the pointer */
}

int a = 5;
actuallyModify(&a);  /* pass the ADDRESS of a */
printf("%d\n", a);  /* now 99 */
```

There is no "pass by reference" in C

What people call pass-by-reference in C is really pass-a-pointer-by-value. The distinction matters: you can reassign the pointer inside the function without affecting the caller, and you must dereference explicitly with `*`. C++ references are a different mechanism that C does not have.

6.3 Returning Multiple Values

A C function returns exactly one value. When you need more, there are three standard approaches.

```
/* 1. Out-parameters: caller supplies addresses to fill in */
void minMax(const float *d, size_t n, float *outMin, float *outMax);

/* 2. Return a struct (Chapter 11) */
typedef struct { float min, max; } range_t;
range_t minMax(const float *d, size_t n);

/* 3. Return a status code, deliver data through out-parameters.
   This is the dominant style in embedded and system code. */
int minMax(const float *d, size_t n, float *outMin, float *outMax);
/* returns 0 on success, negative error code on failure */
```

ODT uses out-parameters plus status codes

ODT signatures such as `void linearForward(layer_t *layer, tensor_t *input, tensor_t *output)` take the destination as a parameter rather than returning it. There are two reasons. First, returning a large struct copies it; passing a pointer does not. Second and more importantly, the caller owns the output buffer — the function never allocates. On a device with 320 KB of SRAM and no malloc during training, every buffer must be allocated once at setup and reused. A function that returned freshly allocated tensors would fragment the heap within a few hundred steps.

6.4 const Parameters

```
/* "I promise not to modify what data points at" */
float meanOf(const float *data, size_t n);

/* ODT: const tensor pointer - the function only reads */
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex);

/* No const - this one writes */
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value);
```

`const` on a parameter documents intent and lets the compiler catch accidental writes. It also lets callers pass `const` data. Note that `const float *p` and `float const *p` mean the same thing — a pointer to constant float. `float * const p` is different: a constant pointer to modifiable float. Chapter 20 covers this fully.

6.5 The Call Stack

Each function call pushes a *stack frame* containing the parameters, the local variables, the return address, and saved registers. When the function returns, the frame is popped and its memory is reused by the next call.

Stack layout during a forward pass

```

Higher addresses
+-----+
| main frame          | locals of main
+-----+
| trainStep frame     | called by main
+-----+
| linearForward frame | called by trainStep
+-----+
| executeOp frame     | called by linearForward
|   + alloca scratch  | <- stack pointer (SP)
+-----+
| (unused stack space)|
|                     |
|                     | v grows down
Lower addresses

```

Stack overflow is silent on a microcontroller

A desktop OS gives each thread megabytes of stack and traps overflow with a guard page. An STM32 has a fixed stack of a few kilobytes carved out of the same 320 KB SRAM as everything else. Overflowing it does not crash — it quietly overwrites your global variables, and the program produces wrong numbers with no indication why. Deep recursion and large local arrays are both dangerous.

```

/* DANGEROUS on an MCU: 4608 bytes on the stack */
void process(void) {
    float window[1152];    /* one HAR window - 4.6 KB! */
    ...
}

/* Safer: static storage, allocated once at link time */
void process(void) {
    static float window[1152]; /* lives in .bss, not the stack */
    ...
}

```

6.6 Recursion

```

unsigned long factorial(unsigned n) {
    if (n <= 1) return 1;          /* base case - must exist */
    return n * factorial(n - 1);   /* recursive case */
}

```

Recursion is elegant for tree and graph problems. It is also a stack consumer: each level adds a frame. On embedded targets, recursion depth must be bounded and provable, or it must be rewritten as a loop. Most safety-critical coding standards (MISRA C, JPL, automotive) ban recursion outright.

```

/* The same computation without recursion - constant stack */
unsigned long factorial(unsigned n) {
    unsigned long result = 1;
    for (unsigned i = 2; i <= n; i++) result *= i;
    return result;
}

```

ODT contains no recursion

Every algorithm in the ODT library is iterative. The tensor shape calculations, the layer traversal, the RigL threshold search — all loops. This is a deliberate constraint: it makes the maximum stack depth statically analysable, which is a hard requirement when the total RAM budget is 320 KB and there is no MMU to catch a runaway.

6.7 Designing Good Functions

Principle	Why
Do one thing	A function named <code>loadAndParseAndValidate</code> should be three functions.
Short	If it does not fit on a screen, it is probably doing too much.
Few parameters	More than about five suggests the parameters want to be a struct.
No hidden state	A function that reads globals is hard to test and hard to reason about. Pass what it needs.
Consistent errors	Pick one convention (0 = success, negative = error) and use it everywhere in the project.
const-correct	Mark every pointer parameter <code>const</code> unless the function writes through it.

Exercises — Chapter 6

1. Write `void swap(int *a, int *b)` and demonstrate it works.
2. Explain why `void swap(int a, int b)` cannot work.
3. Write a function returning both the min and max of a float array via out-parameters.
4. Rewrite the same function to return a struct instead. Which do you prefer, and why?
5. Write a recursive Fibonacci function and an iterative one. Time both for $n=35$. Explain the difference.
6. Write `size_t countActive(const uint8_t *maskBytes, size_t nBits)` that counts set bits in a packed mask.
7. A function declares `float buf[2048];` as a local. How many bytes of stack does it use? Would this fit on an STM32 with an 8 KB stack?

Chapter 7

Arrays

Contiguous storage, indexing, and multi-dimensional layout

7.1 Declaring and Initialising

```
int  a[5];           /* 5 ints, UNINITIALISED (garbage) */
int  b[5] = {1, 2, 3, 4, 5}; /* all five given */
int  c[5] = {1, 2};    /* rest are zero-filled: 1,2,0,0,0 */
int  d[5] = {0};       /* idiom: entire array zeroed */
int  e[] = {1, 2, 3};  /* size deduced: 3 */

float w[9][128];      /* 2D: 9 rows of 128 */
float z[9][128] = {{0}}; /* all 1152 elements zeroed */
```

An array is a block of contiguous memory holding elements of identical type. The size must be known at compile time unless you use a variable-length array (C99, discouraged in embedded code because the size lands on the stack unpredictably).

C never checks array bounds

Writing `a[10]` in a 5-element array compiles cleanly and writes into whatever memory follows the array. This may be another variable, a saved return address, or nothing important — and which one it is can change between builds. Buffer overruns of this kind are the root cause of a large share of all security vulnerabilities ever recorded.

Use `-fsanitize=address` while developing

AddressSanitizer instruments your program to detect out-of-bounds access, use-after-free and memory leaks at runtime, with a clear report naming the file and line. It costs about 2x speed, so it is a development tool, not a release setting — but it finds bugs that would otherwise take days.

7.2 Indexing and the Element-Count Idiom

```
int arr[10];

/* Valid indices are 0 .. 9 */
arr[0] = 1; /* first */
arr[9] = 10; /* last */
arr[10] = 11; /* OUT OF BOUNDS - undefined behaviour */

/* Element count - only works where the array type is visible */
size_t n = sizeof(arr) / sizeof(arr[0]); /* 10 */

/* As a macro */
#define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]))
```


ARRAY_LEN breaks on a pointer

Once an array is passed to a function it becomes a pointer, and `sizeof` gives the pointer size. `ARRAY_LEN` would then compute $8/4 = 2$ regardless of the real length. This is why every C function taking an array also takes a length parameter. Chapter 9 explains the decay rule that causes this.

7.3 Multi-Dimensional Arrays and Row-Major Order

C stores multi-dimensional arrays in *row-major* order: the last index varies fastest. Element `a[i][j]` of an `[R][C]` array sits at flat offset $i * C + j$.

```
int m[3][4] = {
    { 0, 1, 2, 3},
    { 4, 5, 6, 7},
    { 8, 9, 10, 11}
};

/* In memory, one continuous run: */
/* 0 1 2 3 4 5 6 7 8 9 10 11 */

m[1][2] == 11 ... no: offset = 1*4 + 2 = 6, value 6
```

Three dimensions

```
/* HAR batch tensor: [batch][channel][timestep] */
float x[4][9][128];

/* flat offset of x[b][c][t] */
offset = b*(9*128) + c*128 + t;
        = b*1152 + c*128 + t;

/* x[2][5][100] -> 2*1152 + 5*128 + 100 = 2304 + 640 + 100 = 3044 */
```

Why ODT computes flat indices by hand

ODT never declares `float x[4][9][128]`. Tensor shapes are only known at runtime, so the data is a flat `uint8_t *` and the index arithmetic is written out explicitly. This is also what makes *zero-copy transpose* possible: `transposeTensor(t, 0, 1)` swaps two entries in the `orderOfDimensions` array, changing how future index calculations interpret the same bytes, without moving a single byte. A fixed `[4][9][128]` declaration would make that impossible.

7.4 Arrays and Cache Behaviour

```
/* FAST: walks memory in order */
for (size_t i = 0; i < rows; i++)
    for (size_t j = 0; j < cols; j++)
        sum += m[i][j];

/* SLOW: jumps cols*4 bytes on every step */
for (size_t j = 0; j < cols; j++)
    for (size_t i = 0; i < rows; i++)
        sum += m[i][j];
```

Both loops compute the same sum and execute the same number of iterations. On a large matrix the first can be several times faster, because each cache line fetched from memory contains the next several elements the first

loop will need, and none of the elements the second loop will need. The Cortex-M7 has 4 KB of data cache; a 1024-column float matrix has 4 KB per row, so the column-major loop misses on essentially every access.

7.5 Common Array Operations

```
#include <string.h>

memset(arr, 0, sizeof(arr));           /* fill with zero bytes */
memcpy(dst, src, n * sizeof(float));   /* copy, regions must not overlap */
memmove(dst, src, nbytes);             /* copy, overlap allowed */
int r = memcmp(a, b, nbytes);          /* 0 if identical */
```

memset fills BYTES, not elements

`memset(floatArr, 1, n*sizeof(float))` does not set the floats to 1.0 — it sets every byte to 0x01, producing the float value 2.3694e-38. `memset` is only safe for filling with zero, because all-zero bytes happen to be 0.0f and 0 and NULL on every mainstream platform. For any other value, write a loop.

Exercises — Chapter 7

1. Declare a 3x4 int array, fill it with $i*4+j$, and print it as a grid.
2. Write a function that reverses an int array in place.
3. Compute the flat offset of element `[3][7][45]` in a `[4][9][128]` tensor.
4. Time the row-major and column-major summation of a 1024x1024 float matrix. Report the ratio.
5. Write `float dotProduct(const float *a, const float *b, size_t n)`.
6. Explain what `memset(arr, 1, sizeof(arr))` actually does to a float array, and compute the resulting float value.
7. Write a function that zeroes only the elements of an array whose corresponding mask bit is clear, using `tensorBoolGet`-style bit access.

Chapter 8

Pointers: The Core Concept

Addresses, dereferencing, and why pointers exist

8.1 What a Pointer Is

Every byte of memory has a numeric address. A pointer is a variable whose value is such an address. That is the whole idea. The complexity of pointers comes not from the concept but from the syntax and from the discipline required to keep them valid.

```
int x = 42;
int *p = &x;    /* p holds the ADDRESS of x */

printf("%d\n", x);    /* 42          - the value */
printf("%p\n", (void*)&x); /* 0x7ffd... - the address */
printf("%p\n", (void*)p); /* 0x7ffd... - same address */
printf("%d\n", *p);    /* 42          - the value AT that address */
```

Picture it

Memory:

```

address 1000 | 42 | <- x lives here
              +-----+
              ^
              |
address 2000 | 1000 | <- p lives here, holding x's address
              +-----+
```

8.2 The Two Operators

Operator	Name	Meaning
&x	Address-of	Yields a pointer to x
*p	Dereference	Yields the object p points at

The asterisk means two different things

In a **declaration**, `int *p;` means "p is a pointer to int". In an **expression**, `*p` means "the thing p points at". Same symbol, opposite roles. Reading declarations right-to-left helps: `int *p` reads as "p is a pointer to int".

```
int x = 5;
int *p = &x;

*p = 10;    /* writes through p: x is now 10 */
int y = *p; /* reads through p: y is 10 */

int z = 20;
p = &z;    /* p now points at z instead; x untouched */
*p = 30;    /* z is now 30 */
```

8.3 Declaration Syntax Traps

```
int *a, b;      /* a is int*, b is plain int! The * binds to the name. */
int *a, *b;     /* both pointers - what you probably meant */

/* Which is why many style guides put the * with the name: */
int *p;        /* clear that * belongs to p */
int* p;        /* also common, but misleading in multi-declarations */
```

One declaration per line

Declaring several variables on one line saves nothing and creates exactly this class of bug. Write one per line and the ambiguity disappears.

8.4 NULL and Invalid Pointers

```
#include <stddef.h>

int *p = NULL;      /* explicitly points at nothing */

if (p != NULL) {    /* always check before dereferencing */
    printf("%d\n", *p);
}

if (p) { ... }      /* equivalent shorthand - NULL is falsy */
```

Dereferencing NULL is undefined behaviour

On a desktop it usually segfaults, which is merciful because it stops immediately. On an STM32 address 0 is the start of Flash, so `*p` reads a valid byte and the program continues with nonsense data. Writing `*p = 5` to Flash triggers a HardFault. Neither behaviour is guaranteed — the compiler is entitled to assume you never dereference NULL and optimise on that basis, deleting your null checks as dead code.

Three ways a pointer goes bad

Problem	Cause	Defence
Uninitialised	<code>int *p; *p = 5;</code>	<code>p</code> holds stack garbage. Always initialise to NULL or a real address.
Dangling	Points at freed or out-of-scope memory	Set to NULL immediately after free. Never return the address of a local.
Out of bounds	Walked past the end of an array	Keep the length alongside the pointer and check.

```
/* NEVER do this - the local is gone when the function returns */
int *broken(void) {
    int local = 42;
    return &local;    /* dangling pointer */
}

/* Fine - static storage outlives the call */
int *ok(void) {
    static int value = 42;
    return &value;
}
```

8.5 Pointers as Output Parameters

This is the most common everyday use of pointers: letting a function write into the caller's variables.

```
void minMax(const float *data, size_t n, float *outMin, float *outMax) {
    if (n == 0) return;
    float lo = data[0], hi = data[0];
    for (size_t i = 1; i < n; i++) {
        if (data[i] < lo) lo = data[i];
        if (data[i] > hi) hi = data[i];
    }
    *outMin = lo;      /* write through the pointers */
    *outMax = hi;
}

/* Caller */
float lo, hi;
minMax(window, 1152, &lo, &hi);
```

Quantisation calibration uses exactly this

ODT calibrates a quantisation scale by finding the maximum absolute value in a tensor, then computing $\text{scale} = \text{maxAbs} / \text{qMax}$. The min/max scan is an out-parameter function of exactly this shape. The alternative — returning a struct — would work, but out-parameters let the caller reuse existing variables without a copy, and on a Cortex-M7 with no register pressure to spare, that matters.

8.6 const and Pointers

```
const int *p;      /* pointer to const int: cannot do *p = 5 */
int const *p;      /* identical meaning - const binds left */
int * const p;      /* const pointer: cannot do p = &other */
const int * const p; /* neither can change */
```

Read declarations right to left. `int * const p` reads: "p is a const pointer to int". `const int *p` reads: "p is a pointer to int const". The rule is that `const` applies to whatever is immediately to its left, unless there is nothing to its left, in which case it applies to the right.

Mark every read-only pointer parameter const

It costs nothing at runtime, documents the contract, catches accidental writes at compile time, and lets callers pass const data. ODT does this consistently: `tensor_t const *tensor` in the getter, plain `tensor_t *tensor` in the setter. You can tell what a function does to its arguments from the signature alone.

8.7 void Pointers

```
void *generic;      /* points at "some memory", type unknown */

int x = 5;
generic = &x;       /* any pointer converts to void* implicitly */

int *back = generic; /* and back again, in C (a cast is needed in C++) */

/* Cannot dereference or do arithmetic on void* -
   the compiler does not know the element size. */
/* *generic;      ERROR */
/* generic + 1;    ERROR (GCC allows it as an extension, treating it as char*) */
```

`void *` is how C expresses "a pointer to anything". `malloc` returns one. `memcpy` takes them. Callback APIs pass a `void *` context so the callback can receive arbitrary user data.

void* as an operator context in ExecuteOp

ODT's `opSpec_t` holds a kernel function pointer and a `void *ctx`. Each kernel casts the context back to its own parameter struct. This is how a single `executeOp()` function can drive a `matmul`, an elementwise add, and a reduction without knowing anything about their individual parameters. It is C's answer to polymorphism — less safe than a template or a virtual function, but with zero runtime overhead and no vtable in RAM.

Exercises — Chapter 8

1. Declare an int, a pointer to it, and print the value, the address, and the dereferenced pointer.
2. Write `void increment(int *p)` that adds one to the caller's variable.
3. Explain why `int *a, b;` does not declare two pointers.
4. Write a function returning the index of the first negative value in a float array via an out-parameter, returning `true/false` for found/not found.
5. Given `const int * const p`, which of `*p = 1` and `p = &x` are errors?
6. Write a function that takes a `void *` and a type tag, and prints the value as int or float depending on the tag.
7. Explain what is wrong with a function that returns `&local`, and what the symptom would be on an STM32.

Chapter 9

Pointers and Arrays Together

Decay, arithmetic, and the equivalence that is not quite an equivalence

9.1 Array-to-Pointer Decay

In almost every context, an array name evaluates to a pointer to its first element. This is called *decay*, and it is the single most important rule connecting the two concepts.

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;          /* arr decays to &arr[0] - no & needed */

printf("%d\n", arr[0]); /* 10 */
printf("%d\n", *p);    /* 10 - same thing */
printf("%d\n", *arr);  /* 10 - arrays decay in expressions too */
```

The three places decay does NOT happen

Context	Behaviour
<code>sizeof(arr)</code>	Gives the whole array size (20 here), not the pointer size
<code>&arr</code>	Gives a pointer to the <i>array</i> (<code>int (*)[5]</code>), not to the first element
String literal initialiser	<code>char s[] = "hi";</code> copies the characters into <code>s</code> , rather than pointing at them

9.2 Why Functions Lose the Length

```
void f(int arr[5]) {          /* the 5 is IGNORED by the compiler */
    printf("%zu\n", sizeof(arr)); /* 8 - it is really int* */
}

/* These three declarations are identical to the compiler: */
void f(int arr[5]);
void f(int arr[]);
void f(int *arr);
```

This is why every C array function takes a length

The array size vanishes at the function boundary. There is no way to recover it from inside. Consequently the universal C convention is to pass a pointer and a count together: `float meanOf(const float *data, size_t n)`. Forgetting the count, or passing the wrong one, is how buffer overruns happen.

9.3 Pointer Arithmetic

Adding an integer to a pointer advances it by that many *elements*, not bytes. The compiler multiplies by `sizeof` the pointed-to type automatically.

```

int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;          /* points at arr[0] */

p + 1                  /* points at arr[1] - advanced 4 BYTES */
*(p + 2)               /* 30 */
p[2]                   /* 30 - identical; a[i] IS *(a+i) */

p++;                   /* p now points at arr[1] */
printf("%d\n", *p);    /* 20 */

int *q = &arr[4];
printf("%td\n", q - p); /* 3 - difference in ELEMENTS */

```

a[i] is defined as *(a + i)

The subscript operator is pure syntactic sugar. Because addition commutes, `a[i]`, `*(a+i)`, `*(i+a)` and even `i[a]` all mean the same thing. The last one is legal C and occasionally appears in obfuscated code, but never write it.

9.4 Walking an Array With Pointers

```

/* Index style - clearest, and what ODT uses */
for (size_t i = 0; i < n; i++) {
    sum += data[i];
}

/* Pointer style - idiomatic older C */
for (const float *p = data; p < data + n; p++) {
    sum += *p;
}

```

Both compile to nearly identical machine code on any modern compiler. The index form is easier to read and easier to debug, because the loop variable tells you immediately where you are. Prefer it unless you have a specific reason not to.

Rule	Detail
One past the end is legal	You may compute <code>data + n</code> and compare against it. You may not dereference it.
Two past the end is not	Even computing <code>data + n + 1</code> is undefined behaviour, though it will not crash.
Subtraction needs the same array	Subtracting pointers into different arrays is undefined, even though it will produce a number.
void* arithmetic is illegal	The compiler does not know the element size. GCC allows it as an extension, treating void as 1 byte.

9.5 Arrays of Pointers vs Pointers to Arrays

```

int *a[5];          /* array of 5 pointers to int      (5 * 8 = 40 bytes) */
int (*b)[5];        /* pointer to an array of 5 ints   (8 bytes) */

/* Read from the inside out, following normal precedence:
   [] binds tighter than *, so a[5] first -> array of pointers.
   Parentheses force * first -> pointer to array. */

```


Arrays of pointers are common for lists of strings. Pointers to arrays are rare in application code but appear when passing 2D arrays to functions.

```
/* Array of strings */
const char *activities[6] = {
    "walking", "walking_upstairs", "walking_downstairs",
    "sitting", "standing", "laying"
};
printf("%s\n", activities[3]);    /* sitting */
```

9.6 Passing 2D Arrays

```
/* Fixed inner dimension - compiler knows the stride */
void f(int m[][4], size_t rows);
void f(int (*m)[4], size_t rows);    /* identical */

/* Fully dynamic - flatten and index manually.
   This is what ODT does, because shapes are runtime values. */
void g(float *data, size_t rows, size_t cols) {
    for (size_t r = 0; r < rows; r++)
        for (size_t c = 0; c < cols; c++)
            data[r * cols + c] = 0.0f;
}
```

The flat-pointer approach is not a compromise

Manual flat indexing looks less elegant than `m[r][c]`, but it is what makes ODT's tensors work at all. Shapes are runtime data. A tensor can be transposed by permuting a metadata array rather than moving bytes. The same buffer can be reinterpreted as `[4][9][128]` or `[4][1152]` by a Flatten layer with no copy. None of that is expressible with C's built-in multi-dimensional arrays, whose dimensions are baked into the type at compile time.

Exercises — Chapter 9

1. Print `sizeof` an array inside `main` and again inside a function it is passed to. Explain the difference.
2. Write a function summing a float array using pointer arithmetic only, no subscripts.
3. Given `int a[10]; int *p = a + 3;`, what is `p - a`? What is `p[2]` in terms of `a`?
4. Declare an array of 6 string pointers holding the HAR class names and print them in a loop.
5. Explain the difference between `int *a[5]` and `int (*a)[5]`, including the size of each.
6. Write a function that zeroes a rows x cols float matrix passed as a flat pointer.
7. Is `data + n` legal for an n-element array? Is `*(data + n)`? Explain.

Chapter 10

Strings

Null-terminated arrays, the standard functions, and their dangers

10.1 A String Is an Array of char Ending in Zero

C has no string type. A "string" is a convention: a contiguous run of char ending with a byte of value 0, written '\0'. Every standard string function relies on finding that terminator, and every string bug comes from it being missing or misplaced.

```
char s[] = "abc";

/* In memory: 4 bytes, not 3 */
/*  s[0]='a' s[1]='b' s[2]='c' s[3]='\0' */

printf("%zu\n", sizeof(s)); /* 4 - includes the terminator */
printf("%zu\n", strlen(s)); /* 3 - counts up to, not including, \0 */
```

Declaration	Meaning
char s[] = "abc";	Array, 4 bytes, modifiable, on the stack
char *p = "abc";	Pointer to a string literal in read-only memory. Writing through p is undefined behaviour.
const char *p = "abc";	Same, but the compiler now stops you writing. Always prefer this form.

String literals live in read-only memory

On a desktop, char *p = "abc"; p[0] = 'X'; segfaults. On an STM32 the literal is in Flash, so the write silently fails or triggers a fault depending on the memory protection setup. Always declare literal pointers const char * so the compiler catches this at build time.

10.2 The Standard String Functions

Function	Behaviour
strlen(s)	Length excluding \0. O(n) — it scans.
strcpy(d, s)	Copy s to d including \0. No bounds check.
strncpy(d, s, n)	Copy at most n bytes. May not null-terminate.
strcat(d, s)	Append s to d. No bounds check.
strcmp(a, b)	0 if equal, <0 if a<b, >0 if a>b
strncmp(a, b, n)	Compare at most n characters
strchr(s, c)	Pointer to first c, or NULL
strstr(h, n)	Pointer to first occurrence of substring n
strtok(s, delim)	Split on delimiters. Modifies s. Not reentrant.

strcpy and strcat are how buffer overflows happen

Neither knows the size of the destination. `strcpy(small, huge)` writes past the end of `small`, corrupting whatever follows — possibly the saved return address on the stack, which is the classic exploit primitive. Use `snprintf` instead, which always takes a size and always null-terminates.

```
/* Dangerous */
char buf[16];
strcpy(buf, userInput);          /* overflows if input > 15 chars */

/* Safe */
char buf[16];
snprintf(buf, sizeof(buf), "%s", userInput); /* truncates, always terminates */

/* strcpy is NOT the safe version - it may leave buf unterminated */
strcpy(buf, userInput, sizeof(buf));
buf[sizeof(buf) - 1] = '\0';    /* you must do this yourself */
```

10.3 Comparing Strings

```
const char *a = "walking";
const char *b = "walking";

if (a == b) { }                /* WRONG - compares ADDRESSES */
if (strcmp(a, b) == 0) { }     /* Correct - compares CONTENTS */
```

== on strings compares pointers

It may appear to work, because compilers often merge identical string literals so both pointers happen to be equal. Change one character and it breaks. Always use `strcmp`, and remember it returns **0** for equal — the opposite of what a boolean would suggest.

10.4 Parsing Numbers From Text

```
#include <stdlib.h>

long n = strtol(text, NULL, 10); /* base 10 */
float f = strtof(text, NULL);

/* With error detection */
char *end;
errno = 0;
long v = strtol(text, &end, 10);
if (end == text) { /* no digits at all */ }
if (*end != '\0') { /* trailing junk */ }
if (errno == ERANGE) { /* out of range */ }
```

Prefer `strtol` and `strtof` over `atoi` and `atof`. The `ato*` family has no way to report failure: `atoi("hello")` silently returns 0, indistinguishable from `atoi("0")`.

CSVLoader in ODT

ODT's CSV loader reads a line with `fgets`, splits it with `strtok`, and converts each field with `strtof`. The choice of `fgets` over `gets` is not stylistic — `gets` was removed from the language in C11 precisely because it cannot be used safely. `strtok` is acceptable here only because ODT is single-threaded; in threaded code it would have to be `strtok_r`, since `strtok` keeps its position in a hidden static variable.

10.5 Formatted Output

Specifier	Type
%d	int
%u	unsigned int
%ld	long
%zu	size_t
%f	double (float is promoted)
%.2f	double, 2 decimal places
%e	Scientific notation
%s	char* string
%c	single char
%p	pointer (cast to void*)
%x	unsigned in hex
%%	a literal percent sign

printf cannot check your types

It is a variadic function: the compiler cannot verify that the arguments match the format string. `printf("%d", 3.14)` reads the wrong bytes and prints garbage. GCC's `-Wformat` (included in `-Wall`) catches most mismatches for literal format strings, which is one more reason to compile with warnings enabled.

Exercises — Chapter 10

1. Write `size_t myStrlen(const char *s)` without using the standard library.
2. Write a safe copy function that takes a destination size and always null-terminates.
3. Explain why `if (a == b)` is wrong for strings and when it might appear to work.
4. Write a function that counts vowels in a string.
5. Use `strtok` to split "1.5,2.5,3.5" into three floats.
6. Explain what `strncpy(buf, src, sizeof(buf))` does when `src` is exactly `sizeof(buf)` characters long.
7. Write a function mapping a HAR class index 0-5 to its name string, returning `NULL` for invalid input.

Chapter 11

Structures, Unions and Enums

Building your own types

11.1 struct: Grouping Related Data

```
struct point {
    float x;
    float y;
};

struct point p;          /* note: "struct point", not just "point" */
p.x = 1.0f;
p.y = 2.0f;

/* Initialisers */
struct point a = {1.0f, 2.0f};      /* positional */
struct point b = {.x = 1.0f, .y = 2.0f}; /* designated (C99) - preferred */
struct point c = {0};              /* all members zeroed */
```

Use designated initialisers

Positional initialisers silently break when someone reorders the struct members. Designated initialisers name each field, so reordering is harmless and unlisted fields are zeroed. They also document what each value means at the call site.

11.2 typedef: Dropping the struct Keyword

```
typedef struct {
    float x;
    float y;
} point_t;

point_t p = {.x = 1.0f, .y = 2.0f}; /* no "struct" needed */

/* Self-referential structs need a tag, since the typedef is not
   complete until the closing brace */
typedef struct node {
    int value;
    struct node *next; /* must say "struct node" here */
} node_t;
```

11.3 Accessing Members Through a Pointer

```
point_t p = {1.0f, 2.0f};
point_t *ptr = &p;

p.x          /* direct access */
(*ptr).x     /* dereference then access - correct but clumsy */
ptr->x       /* the arrow operator - identical, and idiomatic */
```

The parentheses in `(*ptr).x` are mandatory, because `.` binds tighter than `*`. The arrow operator exists precisely so you never have to write that.

11.4 Nested Structs and Composition

```
/* Simplified from ODT Tensor.h */
typedef struct {
    size_t  numberOfDimensions;
    size_t  *dimensions;      /* e.g. {4, 9, 128} */
    size_t  *orderOfDimensions; /* permutation for zero-copy transpose */
} shape_t;

typedef struct {
    uint8_t  *data;          /* raw bytes - type-agnostic */
    shape_t   *shape;
    quantization_t *quantization; /* how to interpret the bytes */
    sparsity_t  *sparsity;      /* NULL if dense */
} tensor_t;

typedef struct {
    tensor_t *param; /* the weights */
    tensor_t *grad;  /* their gradients, same shape */
} parameter_t;
```

Why these are pointers, not embedded values

ODT stores `shape_t *shape` rather than `shape_t shape`. That allows two tensors to share a shape object, and it allows the Flatten layer to swap a tensor's shape pointer without touching the data. The cost is one indirection per access and the discipline of tracking who owns each allocation — ODT resolves that with an explicit `ownsQuantizations` flag in the layer config, so the destructor knows what to free.

11.5 Padding and Alignment

```
struct bad {
    char a; /* offset 0 */
           /* 3 bytes of padding */
    int b; /* offset 4 - must be 4-byte aligned */
    char c; /* offset 8 */
           /* 3 bytes of padding to make the struct size a multiple of 4 */
}; /* sizeof == 12, though the members total 6 */

struct good {
    int b; /* offset 0 */
    char a; /* offset 4 */
    char c; /* offset 5 */
           /* 2 bytes padding */
}; /* sizeof == 8 */
```

The compiler inserts padding so that each member sits at an address that is a multiple of its size. On ARM Cortex-M, an unaligned 32-bit load either costs extra cycles or faults outright, depending on the instruction. Ordering members from largest to smallest minimises padding — worth doing when a struct is instantiated thousands of times. Chapter 23 covers this in detail.

11.6 union: One Memory, Several Interpretations

```

union value {
    int32_t  asInt;
    float    asFloat;
    uint8_t  asBytes[4];
};

/* sizeof == 4, not 12 - members OVERLAP */

union value v;
v.asFloat = 3.14159f;
printf("%08X\n", v.asInt);      /* the raw bit pattern: 40490FD0 */
printf("%02X\n", v.asBytes[0]); /* lowest byte - depends on endianness */

```

Reading a different member than you wrote is technically undefined

The C standard says only the last-written member has a defined value. In practice every real compiler implements type punning through a union as expected, and C99 explicitly blessed it in a footnote. The portable alternative is `memcpy`, which optimisers reduce to the same instructions.

A more common use is a *tagged union*: a struct containing an enum tag plus a union of possible payloads. Only the member matching the tag is valid.

```

typedef enum { Q_FLOAT32, Q_SYM, Q_ASYM } qtype_t;

typedef struct {
    qtype_t type;          /* the tag says which member is live */
    union {
        struct { float scale; } sym;
        struct { float scale; int32_t zp; } asym;
    } cfg;
} quantization_t;

```

11.7 enum: Named Integer Constants

```

typedef enum {
    ACT_WALKING = 0,
    ACT_WALKING_UPSTAIRS,      /* 1 - auto-increments */
    ACT_WALKING_DOWNSTAIRS,    /* 2 */
    ACT_SITTING,               /* 3 */
    ACT_STANDING,              /* 4 */
    ACT_LAYING,                /* 5 */
    ACT_COUNT                  /* 6 - handy sentinel for array sizes */
} activity_t;

const char *names[ACT_COUNT] = { "walking", ... };

```

The trailing COUNT idiom

Adding a final enumerator named `*_COUNT` gives you the number of real values automatically. Array sizes and loop bounds then update themselves when someone adds a new enumerator, instead of silently going out of sync.

11.8 Passing Structs to Functions

```

void byValue(point_t p);      /* copies the whole struct */
void byPointer(point_t *p);   /* copies only an address */
void readOnly(const point_t *p); /* address, and cannot modify */

```

Small structs (two or three scalars) are cheap to pass by value and the compiler often keeps them in registers. Large structs should be passed by pointer. In embedded code the convention is to pass by pointer almost always, since the stack is scarce and a large copy is both slow and memory-hungry.

Exercises — Chapter 11

1. Define a struct for one HAR sample: a 9x128 float window plus an integer label. What is its size?
2. Write a function that takes a pointer to that struct and returns the mean of all values.
3. Compute `sizeof` for struct `{ char a; int b; char c; }` and explain the padding.
4. Reorder the members to minimise size. What is the new size?
5. Write a union that lets you inspect the four bytes of a float. Print the bytes of 1.0f.
6. Define an enum for the ODT quantisation types with a trailing `COUNT`, and an array of their names indexed by the enum.
7. Explain when you would pass a struct by value rather than by pointer.

Chapter 12

Dynamic Memory

malloc, free, and the discipline they demand

12.1 The Heap

Automatic (stack) variables have a lifetime tied to their block. Static variables last the whole program but their size is fixed at compile time. The heap is the third option: memory you request at runtime, in a size you compute at runtime, that lives until you explicitly release it.

```
#include <stdlib.h>

void *malloc(size_t bytes);           /* uninitialised */
void *calloc(size_t count, size_t size); /* zero-initialised */
void *realloc(void *p, size_t newSize); /* resize */
void free(void *p);                   /* release */

/* Allocate one HAR window */
float *window = malloc(1152 * sizeof(float));
if (window == NULL) {
    /* allocation failed - handle it, do not just continue */
    return -1;
}

window[0] = 0.5f;
/* ... use it ... */

free(window);
window = NULL; /* prevents accidental reuse */
```

sizeof the variable, not the type

Write `malloc(n * sizeof(*window))` rather than `malloc(n * sizeof(float))`. If the type of `window` ever changes, the allocation size follows automatically. This eliminates a whole class of silent bugs.

12.2 The Four Classic Errors

Error	What happens	How to avoid
Memory leak	Allocated and never freed. Usage grows until the program dies.	One owner per allocation. Free on every exit path.
Double free	<code>free()</code> called twice on the same pointer. Corrupts the allocator.	Set the pointer to NULL after freeing. <code>free(NULL)</code> is safe.
Use after free	Dereferencing freed memory. Reads garbage or crashes.	Same discipline: NULL it out immediately.
Buffer overrun	Writing past the allocated size. Corrupts allocator metadata.	Track the size alongside the pointer.

```

/* Leak on the error path - very common */
int bad(void) {
    float *a = malloc(1000);
    float *b = malloc(1000);
    if (b == NULL) return -1;    /* a is LEAKED */
    ...
    free(a); free(b);
    return 0;
}

/* Single-exit cleanup with goto - one of the few good uses of goto */
int good(void) {
    int rc = -1;
    float *a = NULL, *b = NULL;
    a = malloc(1000); if (!a) goto cleanup;
    b = malloc(1000); if (!b) goto cleanup;
    ...
    rc = 0;
cleanup:
    free(a);    /* free(NULL) is a no-op, so this is always safe */
    free(b);
    return rc;
}

```

12.3 Fragmentation

Repeated allocation and release of differently sized blocks leaves the heap pockmarked with small free gaps. Eventually a large request fails even though the total free memory is more than sufficient, because no single gap is big enough. On a desktop with virtual memory this is mostly invisible. On a microcontroller with 320 KB of physically contiguous SRAM, it is fatal.

```

Heap after mixed alloc/free:
[used 100][free 40][used 200][free 30][used 50][free 60]

Total free = 130 bytes.
malloc(100) FAILS - largest contiguous gap is only 60.

```

ODT does not call malloc during training

Every buffer — weights, gradients, activations, optimiser moments, the scratch space used by `executeOp` — is allocated once during setup and reused for every step thereafter. A training loop that allocated per step would fragment the heap within a few hundred iterations and then fail with no useful error. Chapter 21 covers the static-allocation patterns that replace `malloc` entirely.

12.4 Ownership

C has no automatic memory management, so every allocation needs a documented owner: exactly one piece of code responsible for freeing it. Getting this wrong is the source of nearly all memory bugs in large C programs.

Convention	Meaning
Caller allocates, caller frees	Function fills a buffer the caller supplies. Safest. Used throughout ODT.
Callee allocates, caller frees	Function returns a pointer the caller must free. Must be documented loudly.
Callee allocates, callee frees	Function returns a pointer to internal storage. Caller must not free and must not keep it.

The ownsQuantizations flag

ODT's `linearConfig_t` carries a boolean field named `ownsQuantizations`. Some layers create their own quantisation config objects; others share a config owned by the model. The flag records which case applies so the destructor knows whether to free. This is C's answer to a smart pointer: the ownership is data, not language machinery, and you have to maintain it by hand.

Exercises — Chapter 12

1. Allocate an array of `n` floats, fill it with `i*0.5f`, print it, and free it.
2. Write the same function with a deliberate leak, then fix it using the goto-cleanup pattern.
3. Explain why `free(p); free(p);` is dangerous but `free(p); p = NULL; free(p);` is safe.
4. Write a growable int array (start at 4 elements, double on overflow using `realloc`).
5. Draw a heap layout that has 200 bytes free in total but cannot satisfy `malloc(80)`.
6. Compile a leaking program with `-fsanitize=address` and read the report.
7. Explain why ODT allocates all buffers at setup rather than per training step.

Chapter 13

File Input and Output

Streams, text and binary, and reading real datasets

13.1 Opening and Closing

```
#include <stdio.h>

FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    perror("fopen");    /* prints the reason */
    return -1;
}
/* ... use f ... */
fclose(f);
```

Mode	Meaning
"r"	Read text. Fails if the file does not exist.
"w"	Write text. Truncates an existing file to zero.
"a"	Append text. Creates if absent.
"rb" "wb" "ab"	Same, in binary mode
"r+" "w+"	Read and write

The b suffix matters on Windows, where text mode translates `\n` to `\r\n` on write and back on read. For binary data that translation silently corrupts your file. On Linux and macOS the modes are identical, which means bugs of this kind only appear when someone else builds your code.

13.2 Text I/O

```
char line[256];
while (fgets(line, sizeof(line), f) != NULL) {
    /* line includes the trailing newline, if it fit */
    process(line);
}

fprintf(out, "step %zu loss %.4f\n", step, loss);
```

Never use gets()

It has no size parameter and cannot be used safely under any circumstances. It was deprecated in C99 and **removed from the language** in C11. Use `fgets`, which always takes a buffer size and always null-terminates.

13.3 Binary I/O

```

size_t fread (void *ptr, size_t size, size_t count, FILE *f);
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *f);

/* Read one HAR window: 1152 floats */
float window[1152];
size_t got = fread(window, sizeof(float), 1152, f);
if (got != 1152) { /* short read or error */ }

```

Both return the number of *items* transferred, not bytes. A return less than count means end of file or an error — distinguish them with `feof()` and `ferror()`.

13.4 Seeking

```

fseek(f, 0, SEEK_END);      /* jump to end */
long size = ftell(f);       /* current position = file size */
fseek(f, 0, SEEK_SET);     /* back to start */

/* Random access: jump straight to sample number i */
long offset = headerSize + (long)i * 1152 * sizeof(float);
fseek(f, offset, SEEK_SET);
fread(window, sizeof(float), 1152, f);

```

Streaming the HAR dataset

The HAR training set is 6617 windows of 1152 floats: about 30.5 MB. The STM32 has 320 KB of SRAM, so the dataset cannot be loaded. ODT's DataLoader instead keeps a shuffled array of sample indices and, for each one, seeks to `headerSize + index * 4608` and reads exactly one window. Memory cost: one window at a time, 4.6 KB. The seek-and-read pattern above is precisely what makes training on a dataset 95 times larger than RAM possible.

13.5 Parsing the NumPy .npy Format

The HAR data ships as .npy files. The format is simple enough to parse in about forty lines of C, which is why ODT reads it directly rather than converting to a custom format.

npz layout

Offset	Size	Content
0	6	magic: 0x93 'N' 'U' 'M' 'P' 'Y'
6	1	major version (1 or 2)
7	1	minor version
8	2	header length, uint16 little-endian (version 1)
8	4	header length, uint32 little-endian (version 2)
10	L	ASCII dict, padded with spaces to a 64-byte boundary
10+L	...	raw array data, C-contiguous

The header dict for train_x.npy

```

{'descr': '<f4', 'fortran_order': False, 'shape': (6617, 9, 128), }

/* '<' little-endian */
/* 'f' floating point ('\i\' would be int) */
/* '4' four bytes -> float32 */

```

Validating the magic bytes

```
static bool checkMagic(FILE *f) {
    uint8_t magic[6];
    if (fread(magic, 1, 6, f) != 6) return false;
    static const uint8_t expected[6] = {0x93, 'N', 'U', 'M', 'P', 'Y'};
    return memcmp(magic, expected, 6) == 0;
}
```

The leading byte 0x93 is deliberately chosen to be invalid ASCII, so a text editor or a naive text parser immediately recognises the file as binary. Many binary formats use the same trick: PNG starts with 0x89, and ELF with 0x7F.

Exercises — Chapter 13

1. Write a program that counts the lines in a text file.
2. Write a program that copies a file byte by byte using `fread` and `fwrite`.
3. Determine a file's size using `fseek` and `ftell`.
4. Write a function that reads sample `i` from a flat binary file of 1152-float records.
5. Write the magic-byte check for a `.npy` file and test it on a real one.
6. Explain what goes wrong if you open a binary file in text mode on Windows.
7. Write a function that appends a line to a log file, creating it if it does not exist.

Chapter 14

The Preprocessor

Text substitution before the compiler ever runs

14.1 What the Preprocessor Is

The preprocessor is a separate program that runs before the compiler. It understands nothing about C types, scopes or expressions — it manipulates text. This makes it powerful and dangerous in equal measure. Run `gcc -E file.c` to see exactly what the compiler receives.

14.2 #define

```
#define MAX_LAYERS 16
#define HAR_CLASSES 6

/* Function-like macro - ALWAYS parenthesise everything */
#define SQUARE(x) ((x) * (x))
#define MIN(a, b) ((a) < (b) ? (a) : (b))
```

Missing parentheses break macros

`#define SQUARE(x) x*x` expands `SQUARE(1+2)` into `1+2*1+2`, which is 5, not 9. Parenthesise every parameter **and** the whole body. This is not defensive style; it is required for correctness.

Macros evaluate arguments more than once

`MIN(i++, j)` increments `i` once or twice depending on which branch wins. Never pass an expression with side effects to a macro. A static `inline` function is safer and just as fast.

```
static inline int minInt(int a, int b) {
    return a < b ? a : b;
}
/* Type-checked, single evaluation, and the compiler inlines it anyway. */
```

14.3 The do-while(false) Idiom

```
/* Broken: two statements, so the if only covers the first */
#define LOG_BAD(m) printf("%s", m); fflush(stdout)
if (verbose) LOG_BAD("hi"); /* fflush runs unconditionally! */

/* Correct: one statement; trailing semicolon works naturally */
#define LOG(m) do { printf("%s", m); fflush(stdout); } while (false)
if (verbose) LOG("hi");
else          other(); /* else still binds correctly */
```

The `while (false)` is optimised away entirely. The construct exists purely to make a multi-statement macro behave like a single statement in every syntactic context.

ODT debug macros

```
#define PRINT_INFO(fmt, ...) do { if (DLEVEL >= 2) { printf(fmt, ##__VA_ARGS__); } } while (false)
```

Three ideas at once. The do-while makes it a single statement. DLEVEL is a compile-time constant, so when it is 0 the compiler deletes the entire body as dead code — zero runtime cost, zero code size. And `##__VA_ARGS__` is the GCC extension that deletes the preceding comma when no variadic arguments are supplied, so `PRINT_INFO("done")` compiles as well as `PRINT_INFO("step %d", n)`.

14.4 Conditional Compilation

```
#ifdef DEBUG
    printf("debug build\n");
#endif

#if DLEVEL >= 2
    /* included only when DLEVEL is 2 or more */
#elif DLEVEL == 1
#else
#endif
```

Values are supplied on the command line with `-D`: `gcc -DDLEVEL=2 ...`. This is how ODT selects debug verbosity at build time with no runtime branch.

14.5 Include Guards

```
#ifndef ODT_TENSOR_H
#define ODT_TENSOR_H
    ... declarations ...
#endif /* ODT_TENSOR_H */
```

Without a guard, a header included twice would define its types twice, which is an error. `#pragma once` does the same in one line and is supported everywhere, though it is not standard C.

14.6 Stringify and Token Paste

```
#define STR(x)    #x          /* turns x into a string literal */
#define CAT(a,b) a##b        /* glues two tokens together */
STR(hello)      -> "hello"
CAT(my, Var)    -> myVar

/* Two-level indirection to stringify a macro VALUE */
#define XSTR(x)  STR(x)
#define VERSION  3
STR(VERSION)    -> "VERSION" /* not what you wanted */
XSTR(VERSION)   -> "3"       /* correct */
```

14.7 Predefined Macros

Macro	Expands to
<code>__FILE__</code>	Current source file name, as a string

Macro	Expands to
<code>__LINE__</code>	Current line number, as an int
<code>__func__</code>	Current function name
<code>__DATE__</code> / <code>__TIME__</code>	Compilation date and time
<code>__STDC_VERSION__</code>	201112L for C11, 199901L for C99

Error macro with location

```
#define PRINT_ERROR(fmt, ...) \
do { if (DLEVEL >= 1) \
    fprintf(stderr, "[%s:%d %s] " fmt "\n", \
        __FILE__, __LINE__, __func__, ##__VA_ARGS__); \
} while (false)
```

A macro ends at the newline unless continued with a backslash. Note also that C concatenates adjacent string literals at compile time, which is how the caller format string is spliced into the middle.

Exercises — Chapter 14

1. Write a `MAX(a, b)` macro with correct parenthesisation.
2. Show an argument for which an unparenthesised `SQUARE` gives the wrong answer.
3. Write a `SWAP(a, b, type)` macro using the do-while idiom.
4. Explain why `MIN(i++, j)` is dangerous and what to use instead.
5. Write an assertion macro printing file, line, function and the failing expression.
6. Add include guards to a header and show that including it twice compiles.
7. Run `gcc -E` on a file with three macros and find their expansions.

Chapter 15

Multi-File Programs

Headers, translation units, linkage and build systems

15.1 Translation Units

Each .c file is compiled independently into an object file. The compiler sees nothing of the other files. Only the linker, at the very end, connects a call in one object file to a definition in another. Understanding this split explains most confusing C build errors.

Stage	Typical message	Meaning
Compile	implicit declaration of function 'f'	You called f without including its header
Compile	unknown type name 'tensor_t'	Missing #include for the type
Link	undefined reference to 'f'	Declared but never defined, or the .c file was not compiled in
Link	multiple definition of 'x'	A variable was defined in a header, or in two .c files

15.2 What Goes in a Header

```
/* rng.h - the interface */
#ifndef ODT_RNG_H
#define ODT_RNG_H
#include <stdint.h>
#include <stddef.h>

void    rngSeed(uint32_t seed);
uint32_t rngNext(void);
uint32_t rngBounded(uint32_t bound);
float   rngFloat(void);
#endif
```

```
/* rng.c - the implementation */
#include "rng.h"

static uint32_t state = 1;    /* PRIVATE to this file */

void rngSeed(uint32_t seed) {
    state = seed ? seed : UINT32_MAX;
}

uint32_t rngNext(void) {
    uint32_t x = state;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    state = x;
    return x;
}
```

Put in the header	Keep in the .c file
Function prototypes	Function bodies
Type definitions (struct, enum, typedef)	static helper functions
Macros used by callers	static file-scope variables
extern declarations of globals	Definitions of globals
static inline functions	Everything else

Never define a variable in a header

Writing `int counter;` in a header creates one copy in every .c file that includes it, and the linker reports "multiple definition". Declare it `extern int counter;` in the header and define it exactly once in a single .c file.

15.3 Linkage

Declaration	Linkage	Visible to
<code>int x;</code> at file scope	External	Every translation unit
<code>static int x;</code> at file scope	Internal	This file only
<code>extern int x;</code>	External	Refers to a definition elsewhere
<code>void f(void)</code>	External	Every translation unit
<code>static void f(void)</code>	Internal	This file only

static is C's private keyword

Marking every function and variable `static` unless it is part of the module public interface gives real encapsulation, and lets the compiler optimise harder because nothing outside the file can call or modify it. ODT keeps its RNG state `static uint32_t state;` for exactly this reason.

15.4 Building

```
# Compile each unit separately, then link
gcc -c -Wall -std=c11 rng.c -o rng.o
gcc -c -Wall -std=c11 tensor.c -o tensor.o
gcc -c -Wall -std=c11 main.c -o main.o
gcc rng.o tensor.o main.o -lm -o program

# -lm links the math library (sqrtf, cosf, expf)
```

A minimal Makefile

```

CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -O2 -Iinclude
LDFLAGS = -lm
OBJS    = rng.o tensor.o linear.o adamw.o main.o

program: $(OBJS)
■$(CC) $(OBJS) $(LDFLAGS) -o $@

%.o: %.c
■$(CC) $(CFLAGS) -c $< -o $@

clean:
■rm -f $(OBJS) program

.PHONY: clean

```

Make rebuilds only what changed, by comparing timestamps. `%.o: %.c` is a pattern rule; `$@` is the target and `$<` the first prerequisite. Make requires real tab characters for indentation, not spaces.

15.5 Organising a Project

Layout resembling ODT

```

odt/
  include/          public headers, one per module
    rng.h  tensor.h  layer.h
  src/
    rng/RNG.c
    tensor/Tensor.c
    layer/Linear.c
    optimizer/AdamW.c
  examples/har_classifier/
  tests/
  Makefile

```

One module means one header plus one or more `.c` files. The header defines the interface; anything not in the header should be static. Keeping public headers in a separate `include/` directory makes the interface boundary physical rather than a matter of convention.

Exercises — Chapter 15

1. Split a single-file program into `math.c`, `math.h` and `main.c`, and build it.
2. Trigger an "undefined reference" error deliberately, then fix it.
3. Trigger a "multiple definition" error by defining a variable in a header, then fix it with `extern`.
4. Write a Makefile for a three-file project with a clean target.
5. Mark a helper static and confirm another file cannot call it.
6. Explain which errors appear at compile time and which at link time, and why.
7. Add include guards to two headers where one includes the other, and verify no error.

Chapter 16

The Standard Library

What is available, and what to use it for

16.1 The Headers Worth Knowing

Header	Provides
<stdio.h>	printf, fopen, fread, fgets, snprintf
<stdlib.h>	malloc, free, strtol, strtouf, qsort, bsearch, exit
<string.h>	strlen, strcmp, memcpy, memset, memmove, strtok
<math.h>	sqrtf, expf, logf, sinf, cosf, fabsf, roundf, powf
<stdint.h>	int8_t..uint64_t, INT32_MAX, UINT32_MAX
<stddef.h>	size_t, ptrdiff_t, NULL, offsetof
<stdbool.h>	bool, true, false
<limits.h>	INT_MAX, CHAR_BIT, SIZE_MAX
<float.h>	FLT_EPSILON, FLT_MAX, DBL_EPSILON
<assert.h>	assert (removed by -DNDEBUG)
<time.h>	clock, time, difftime

16.2 math.h: Use the f Suffix

```
float a = sqrtf(2.0f); /* float version - one instruction on the Cortex-M7 FPU */
double b = sqrt(2.0); /* double version - software-emulated, far slower */

fabsf(x)    expf(x)    logf(x)    powf(x, y)
sinf(x)     cosf(x)    roundf(x) floorf(x) ceilf(x)
fminf(a,b)  fmaxf(a,b) fmodf(a,b)
```

Omitting the f promotes to double

Writing `sqrt(x)` where `x` is a float converts to double, computes in double, converts back. The Cortex-M7 has a single-precision FPU; double-precision operations are emulated in software and can be twenty times slower. In numeric embedded code this one omission has been known to halve throughput.

16.3 qsort and bsearch

```
static int cmpFloat(const void *a, const void *b) {
    float x = *(const float *)a;
    float y = *(const float *)b;
    /* Do NOT return (int)(x - y): it truncates and can overflow */
    return (x > y) - (x < y);
}

qsort(arr, n, sizeof(float), cmpFloat);
float key = 3.5f;
float *found = bsearch(&key, arr, n, sizeof(float), cmpFloat);
```

The comparator receives `void *` and must cast. The idiom `(x > y) - (x < y)` yields -1, 0 or 1 with no subtraction, avoiding both integer overflow and float truncation.

Why ODT does not call `qsort` for `RigL`

The `RigL` drop step needs the K -th smallest weight magnitude. A full `qsort` is $O(n \log n)$ and needs a scratch copy — for a 524288-element layer that is 2 MB, which does not exist on the device. ODT instead runs a partial selection sort that stops after K passes: $O(n \cdot K)$ time and only the active-weight buffer. When K is a few percent of n that is both faster and vastly smaller. Choosing an algorithm by its memory profile rather than its asymptotic time is characteristic of embedded work.

16.4 assert

```
#include <assert.h>
assert(ptr != NULL);
assert(index < arrayLength);
/* Compiling with -DNDEBUG removes every assert entirely */
```

Never put side effects in an assert

`assert(readSensor() == 0);` stops calling `readSensor` in release builds, because the whole expression disappears. Asserts check invariants that should be impossible to violate. Real error conditions need real error handling.

16.5 Timing

```
#include <time.h>
clock_t t0 = clock();
doWork();
clock_t t1 = clock();
double secs = (double)(t1 - t0) / CLOCKS_PER_SEC;
printf("%.3f s\n", secs);
```

On an MCU there is no `clock()`. You read a hardware cycle counter instead — on Cortex-M7 that is the DWT CYCCNT register, covered in Chapter 27.

Exercises — Chapter 16

1. Sort an array of 1000 random floats with `qsort` and verify it is ordered.
2. Explain why `return (int)(x - y);` is a broken float comparator.
3. Time two implementations of the same loop using `clock()`.
4. Rewrite a numeric function using `sqrt/exp` to use the float versions, and measure the difference.
5. Use `bsearch` to find a value in a sorted array.
6. Write an assert checking a tensor index is in range; show it disappears under `-DNDEBUG`.
7. Look up `FLT_EPSILON` and write a relative-tolerance float comparison.

PART II

Embedded and Systems C

The same language under pressure: 320 KB of RAM, no operating system, no safety net. Every technique here exists because a constraint forced it.

Chapter 17

Bit Manipulation

Packing, masking, and addressing individual bits

17.1 Why Bits Matter

On a desktop, storing a boolean in a whole byte wastes 7 bits and nobody notices. On a device with 320 KB of SRAM holding a neural network, the same waste is the difference between a model that fits and one that does not. A sparsity mask over 524288 weights costs 512 KB at one byte per weight, or 64 KB packed at one bit per weight. Only the second fits.

17.2 The Four Fundamental Idioms

```
/* TEST bit n */
bool isSet = (value >> n) & 1u;
bool isSet = (value & (1u << n)) != 0;      /* equivalent */

/* SET bit n to 1 */
value |= (1u << n);

/* CLEAR bit n to 0 */
value &= ~(1u << n);

/* TOGGLE bit n */
value ^= (1u << n);
```

Every bit operation in every embedded codebase is built from these four. Understand why each works and you can read any register-manipulation code.

Why CLEAR needs the NOT

```
n = 4
1u << 4 = 0b00010000 /* only bit 4 */
~(1u << 4) = 0b11101111 /* every bit EXCEPT bit 4 */

value = 0b10110100
& mask = 0b11101111
      = 0b10100100 /* bit 4 cleared, all others preserved */
```

17.3 Bit-Packed Arrays

To store N booleans in $\text{ceil}(N/8)$ bytes, element i lives at bit $i/8$ of byte $i/8$. The division and modulo are cheap because 8 is a power of two — the compiler turns them into a shift and a mask.

```
Flat index: 0 1 2 3 4 5 6 7 | 8 9 10 11 12 13 14 15
              \         /         \         /
              data[0]             data[1]

index 15 -> byteIndex = 15/8 = 1, bitIndex = 15%8 = 7
           -> bit 7 (the MSB) of data[1]
```

ODT Tensor.c - the real implementation

```

bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {
    if (tensor->quantization->type != BOOL) {
        PRINT_ERROR("tensorBoolGet called on non-BOOL tensor");
        exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;
}

void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {
    if (tensor->quantization->type != BOOL) {
        PRINT_ERROR("tensorBoolSet called on non-BOOL tensor");
        exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    if (value) {
        tensor->data[byteIndex] |= (uint8_t)(1u << bitIndex);
    } else {
        tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex);
    }
}

```

Reading this code

The getter is the TEST idiom; the setter is SET and CLEAR. The (uint8_t) casts exist because 1u << bitIndex is a 32-bit unsigned int — casting back to 8 bits makes the narrowing explicit and silences -Wconversion. The type guard at the top costs one comparison and catches the case where a caller passes a float tensor, which would otherwise read meaningless bits with no symptom until the numbers came out wrong.

```

/* Allocating a packed mask: ceiling division */
size_t byteCount = (numWeights + 7) / 8;
uint8_t *maskData = calloc(byteCount, 1); /* calloc zeroes -> all inactive */

/* 524288 weights -> (524288 + 7)/8 = 65536 bytes = 64 KB */

```

17.4 Bit Fields Within a Byte

Sub-byte quantisation packs several small values into one byte. Extracting a field requires a mask for the range and a shift to right-align it.

ODT sub-byte field access

```

uint32_t getBitmask(uint32_t startbit, uint32_t endbit) {
    uint32_t endbitInternal = endbit - (startbit / 8) * 8;
    uint32_t startbitInternal = startbit - (startbit / 8) * 8;
    uint32_t counter = 0;
    uint32_t value = 1;
    for (size_t i = 0; i < 8; i++) {
        if ((i >= startbitInternal) & (endbitInternal > i)) {
            counter += value;
        }
        value *= 2;
    }
    return counter;
}

uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit) {
    uint8_t bitmask = getBitmask(startbit, endbit);
    uint8_t intermediate = data & bitmask; /* isolate the field */
    intermediate >>= startbit - (startbit / 8) * 8; /* right-align it */
    return intermediate;
}

```

Tracing getBitmask(11, 14)

```

(startbit / 8) * 8 = (11/8)*8 = 8 /* byte 1 begins at global bit 8 */
startbitInternal = 11 - 8 = 3
endbitInternal = 14 - 8 = 6

i=0 value=1 3<=0? no -> skip
i=1 value=2 3<=1? no -> skip
i=2 value=4 3<=2? no -> skip
i=3 value=8 3<=3 yes, 6>3 yes -> counter += 8 -> 8
i=4 value=16 3<=4 yes, 6>4 yes -> counter += 16 -> 24
i=5 value=32 3<=5 yes, 6>5 yes -> counter += 32 -> 56
i=6 value=64 6>6? no -> skip
i=7 value=128 6>7? no -> skip

result = 56 = 0b00111000 /* bits 3,4,5 - exactly [11,14) */

```

Note that value holds 2^i at iteration i , because the doubling happens *after* the test. That is precisely the bit weight of position i : bit 0 has weight 1, bit 3 has weight 8, bit 7 has weight 128.

Tracing readByte(0b10110100, 11, 14)

```

bitmask = 0b00111000
data & mask = 0b10110100 & 0b00111000 = 0b00110000
shift amount = 11 - 8 = 3
result = 0b00110000 >> 3 = 0b00000110 = 6

/* Check by hand: bits 5,4,3 of data are 1,1,0 -> binary 110 = 6 */

```

The half-open convention

Both functions treat the range as $[startbit, endbit)$ — start included, end excluded. This is the same convention as array slicing in most languages, and it makes the length simply $endbit - startbit$ with no off-by-one adjustment. It also means adjacent fields share a boundary value: $[0,3)$ and $[3,6)$ are contiguous and non-overlapping.

getBitmask assumes the field is inside one byte

If `endbit` crosses into the next byte, the loop only runs to `i=7` and the mask is silently clipped. A field straddling a byte boundary must be split by the caller into two calls and recombined. ODT's packing layout is arranged so this never happens.

17.5 Counting Bits

```
/* Naive: 8 iterations per byte */
size_t popcount8(uint8_t b) {
    size_t c = 0;
    for (int i = 0; i < 8; i++) c += (b >> i) & 1u;
    return c;
}

/* Kernighan's trick: iterates once per SET bit, not per bit */
size_t popcount32(uint32_t v) {
    size_t c = 0;
    while (v) { v &= v - 1; c++; } /* clears the lowest set bit */
    return c;
}

/* Best: the hardware instruction, if available */
size_t c = __builtin_popcount(v); /* GCC/Clang */
```

`v & (v - 1)` clears the lowest set bit because subtracting 1 flips that bit to 0 and sets all bits below it to 1; the AND then wipes the whole low run. For a mask that is 80 percent zeros, this runs five times faster than the naive loop.

Counting active weights in RigL

Every RigL step needs the number of currently active weights, to compute $K = \alpha * \text{numActive}$. The straightforward version calls `tensorBoolGet` once per weight — 524288 calls, each with a division, a modulo, a shift and a mask. Iterating over the 65536 packed bytes and calling `__builtin_popcount` on each is roughly eight times faster and gives an identical answer.

17.6 Useful Bit Tricks

Expression	Effect
<code>v & (v-1)</code>	Clear the lowest set bit
<code>v & -v</code>	Isolate the lowest set bit
<code>v (v+1)</code>	Set the lowest clear bit
<code>(v & (v-1)) == 0</code>	True if v is a power of two (or zero)
<code>v & (n-1)</code>	Equivalent to <code>v % n</code> when n is a power of two
<code>(v + 7) & ~7u</code>	Round v up to the next multiple of 8
<code>a ^ b ^ a</code>	Equals b — the basis of XOR swap

Shift rules you must not break

Shifting by an amount greater than or equal to the type width is undefined behaviour, not zero. `1u << 32` on a 32-bit int is UB. Shifting by a negative amount is UB. Right-shifting a negative signed value is implementation-defined. Always shift unsigned types, and always check the shift amount if it is a variable.

Exercises — Chapter 17

1. Write functions to set, clear, toggle and test bit `n` of a `uint32_t`.
2. For flat index 23, compute the byte index and bit index. Which bit of which byte?
3. Trace `getBitmask(2, 6)` by hand and give the binary result.
4. Trace `readByte(0b11010110, 2, 6)` and give the decimal result.
5. Implement `writeByte`: the inverse of `readByte`, inserting a right-aligned field into a byte at a given bit range.
6. Count set bits in a 64 KB mask using both the naive loop and `__builtin_popcount`. Time both.
7. Explain why `v & (v-1)` clears the lowest set bit, working through the binary for `v = 0b10110000`.
8. Write a ceiling-division macro `CEIL_DIV(a, b)` and use it to compute the byte count for a 1000-bit mask.

Fixed-Point Arithmetic

Representing fractions with integers

18.1 Why Not Just Use Float

Many microcontrollers have no floating-point unit at all. On those, every float operation is a software routine costing tens to hundreds of cycles. Even on the Cortex-M7, which has a single-precision FPU, integer arithmetic is smaller in code size and consumes less energy. And for neural network inference, 8-bit integers give a 4x memory saving over float32 with negligible accuracy loss.

18.2 The Core Idea

A fixed-point number stores an integer *mantissa* together with an implied *scale*. The real value is their product.

```
real = mantissa * scale

/* Example: scale = 0.01, so the mantissa counts hundredths */
mantissa = 314  ->  real = 3.14
mantissa = -50  ->  real = -0.50

/* The scale is NOT stored per value - one scale covers a whole tensor */
```

Property	Floating point	Fixed point
Scale	Per value, in the exponent field	Shared, known at compile or calibration time
Spacing	Relative — grows with magnitude	Absolute — uniform across the range
Range	Enormous (10^{38})	Narrow: $qMax * scale$
Cost	FPU instruction, or software emulation	Plain integer add/multiply
Storage	4 bytes minimum	1 byte for int8

18.3 Symmetric Quantisation

```
/* Quantise: real -> mantissa */
int8_t quantise(float real, float scale) {
    float scaled = real / scale;
    int32_t r = (int32_t)roundf(scaled);
    if (r < -128) r = -128;          /* clamp before narrowing */
    if (r > 127) r = 127;
    return (int8_t)r;
}

/* Dequantise: mantissa -> real */
float dequantise(int8_t q, float scale) {
    return (float)q * scale;
}
```

Choosing the scale

```

/* Map the largest magnitude in the tensor to the largest integer */
float maxAbs = 0.0f;
for (size_t i = 0; i < n; i++) {
    float a = fabsf(data[i]);
    if (a > maxAbs) maxAbs = a;
}
float scale = maxAbs / 127.0f;

```

Quantity	Formula	Example (maxAbs = 2.54)
scale	$\text{maxAbs} / \text{qMax}$	$2.54 / 127 = 0.02$
Resolution (LSB)	scale	0.02
Representable range	$[-\text{qMax} * \text{scale}, +\text{qMax} * \text{scale}]$	$[-2.54, +2.54]$
Max rounding error	$\text{scale} / 2$	0.01
Mean squared error	$\text{scale}^2 / 12$	0.000033

18.4 Asymmetric Quantisation

Symmetric quantisation wastes half its range on a tensor that is never negative — a ReLU output, for instance. Asymmetric quantisation adds an integer offset so the representable window can sit anywhere.

```

real = (mantissa - zeroPoint) * scale

/* For data in [0, 6.0] using uint8 mantissas 0..255: */
scale      = (6.0 - 0.0) / 255 = 0.0235
zeroPoint = 0

/* real 0.0 -> mantissa 0; real 6.0 -> mantissa 255 */
/* Every one of the 256 codes is used. Symmetric would waste 128 of them. */

```

zeroPoint is int32 in ODT, not uint8

It is tempting to store the zero point as a `uint8_t`, since it is conceptually a mantissa value. ODT declares it `int32_t` because the quantisation code supports up to 30 bits, and at 30 bits the zero point can be far outside the 0..255 window. Sizing a field to the common case and then discovering it overflows in the general case is a classic embedded bug; ODT pays 3 extra bytes per quantisation config to avoid it entirely.

18.5 Accumulator Width

Multiplying two `int8` values gives at most $127 * 127 = 16129$, which needs 15 bits. Summing many such products needs more. Choosing the accumulator width is a matter of counting worst cases.

```

Worst-case single product: 127 * 127          = 16,129
Sum of N products:        N * 16,129

int16 accumulator holds 32,767      -> safe for N = 2
int32 accumulator holds 2,147,483,647 -> safe for N = 133,169

/* Typical layer widths are under 4096, so int32 has a 32x margin. */

```

```
int32_t acc = 0;
for (size_t k = 0; k < K; k++) {
    acc += (int32_t)a[k] * (int32_t)b[k];    /* cast BEFORE multiplying */
}

/* Without the casts, int8 * int8 promotes to int, which is fine here -
   but writing them makes the intent explicit and survives a type change. */
```

The accumulator has its own scale

If the input has scale s_x and the weights scale s_w , then the accumulator represents real values with scale $s_x * s_w$. To write the result back as int8 with output scale s_{out} , multiply by $s_x * s_w / s_{out}$, round, and clamp. ODT calls this `rescaleIntoAccumulatorScale`, and it is the reason the `SYM_INT32` quantisation type exists.

18.6 Stochastic Rounding

Deterministic rounding has a fatal flaw when training in low precision: any gradient smaller than half an LSB rounds to zero, every single time. The weight never moves. With int8 gradients and a small learning rate, most of the network stops learning.

```
/* Deterministic - the dead zone */
grad = 0.3 * scale    ->  round(0.3) = 0    -> weight never updates

/* Stochastic - add uniform noise first */
float roundSRHalfAway(float input) {
    return roundHalfAway(input + rngNextFloat() - 0.5f);
}

grad = 0.3 * scale    ->  rounds to 1 with probability 0.3
                        ->  rounds to 0 with probability 0.7
                        ->  expected value = 0.3    (exactly correct)
```

Why the expectation is exact

Let x be in $[n, n+1)$ for some integer n , and let $U \sim \text{Uniform}[0,1)$.

$x + U - 0.5$ lies in $[n - 0.5, n + 1.5)$

$$\text{round}(x + U - 0.5) = \begin{cases} n & \text{when } x + U - 0.5 < n + 0.5 \\ & \text{i.e. } U < n + 1 - x \\ n + 1 & \text{otherwise} \end{cases}$$

$$\begin{aligned} \text{prob} &= n + 1 - x \\ \text{prob} &= x - n \end{aligned}$$

$$\begin{aligned} E[\text{result}] &= n(n + 1 - x) + (n + 1)(x - n) \\ &= n^2n + n - n^2x + n^2x - n^2n + x - n \\ &= x \end{aligned}$$

QED

Stochastic rounding is what makes FQT work

The FQT paper this project builds on quantises the backward pass as well as the forward pass. Without stochastic rounding, quantised gradients collapse to zero and training stalls within a few epochs. ODT's `SR_HALF_AWAY` mode is used for weight and gradient conversion, while activations use plain deterministic rounding — activations are large enough that the dead zone does not bite, and deterministic rounding is one instruction cheaper.

18.7 Q Notation

```
Qm.n means m integer bits and n fractional bits (plus a sign bit)

Q7.8 in an int16: 1 sign + 7 integer + 8 fractional
scale = 2-8 = 1/256
range = [-128.0, 127.996]
resolution = 0.0039

/* Conversion is just a shift, because the scale is a power of two */
int16_t toQ7_8(float f) { return (int16_t)(f * 256.0f); }
float fromQ7_8(int16_t q){ return (float)q / 256.0f; }

/* Multiplication doubles the fractional bits, so shift back down */
int16_t mulQ7_8(int16_t a, int16_t b) {
    int32_t p = (int32_t)a * b;      /* now Q15.16 */
    return (int16_t)(p >> 8);      /* back to Q7.8 */
}
```

Power-of-two scales make conversion a shift rather than a divide, which matters enormously on a CPU without a hardware divider. ODT uses arbitrary float scales instead, trading that speed for the ability to fit the scale tightly to the actual data range — on a Cortex-M7 with an FPU, the multiply is cheap enough that the better range utilisation wins.

Exercises — Chapter 18

1. A tensor has `maxAbs = 5.08`. Compute the `int8` symmetric scale, the resolution, and the maximum rounding error.
2. Quantise 3.7 and -1.2 with that scale, then dequantise them. What is the error in each?
3. Write `quantise` and `dequantise` functions with correct clamping.
4. For a 1024-element dot product of `int8` values, what is the largest possible accumulator value? Does it fit in `int32`?
5. Implement stochastic rounding and verify empirically that the mean of 100000 roundings of 0.3 is close to 0.3.
6. Convert 3.14159 to Q7.8 and back. What is the round-trip error?
7. Explain in one paragraph why deterministic rounding stalls quantised training.
8. A ReLU output ranges over [0, 6]. Compute the asymmetric `uint8` scale and zero point.

Chapter 19

Memory Layout

Flash, SRAM, stack, heap, and the linker script

19.1 The STM32 Nucleo-F746ZG Memory Map

Region	Size	Speed	Contents
Flash	1 MB	Slower; cached by the ART accelerator	Code, const data, literals
SRAM1	240 KB	Single cycle	Globals, heap, stack
SRAM2	16 KB	Single cycle	Usually a second pool
DTCM	64 KB	Zero wait state, tightly coupled	Hot data, stack
ITCM	16 KB	Zero wait state	Hot code

Total usable RAM is 320 KB. The model weights, their gradients, the AdamW moment tensors, the activation buffers, the stack and any heap all have to coexist inside that number.

19.2 The Linker Sections

Section	Lives in	Holds
.text	Flash	Machine code
.rodata	Flash	const data, string literals, lookup tables
.data	Flash then copied to SRAM at boot	Initialised globals and statics
.bss	SRAM, zeroed at boot	Uninitialised globals and statics
heap	SRAM	malloc allocations, grows upward
stack	SRAM	Locals and call frames, grows downward

```
const float table[256] = { ... }; /* .rodata - Flash, 0 SRAM */
int counter = 42; /* .data - 4 bytes SRAM + 4 bytes Flash */
int buffer[1000]; /* .bss - 4000 bytes SRAM, no Flash cost */
static float weights[512]; /* .bss - 2048 bytes SRAM */

void f(void) {
    float local[128]; /* stack - 512 bytes, transient */
    float *p = malloc(1024); /* heap - 1024 bytes until freed */
}
```

const is free SRAM

Marking a lookup table const moves it from .data (SRAM plus a Flash copy) to .rodata (Flash only). A 256-entry float table costs 1 KB of SRAM without const and zero with it. On a 320 KB budget that is a meaningful saving for a one-word change.

19.3 Why .data Costs Twice

An initialised global needs its initial value stored somewhere non-volatile, because SRAM is empty at power-on. So the value sits in Flash, and the startup code copies it into SRAM before main runs. The variable therefore consumes both.

```
/* Simplified startup sequence */
void Reset_Handler(void) {
    /* 1. Copy .data from Flash to SRAM */
    uint8_t *src = &_sidata;          /* Flash source, from the linker script */
    uint8_t *dst = &_sdata;          /* SRAM destination */
    while (dst < &_edata) *dst++ = *src++;

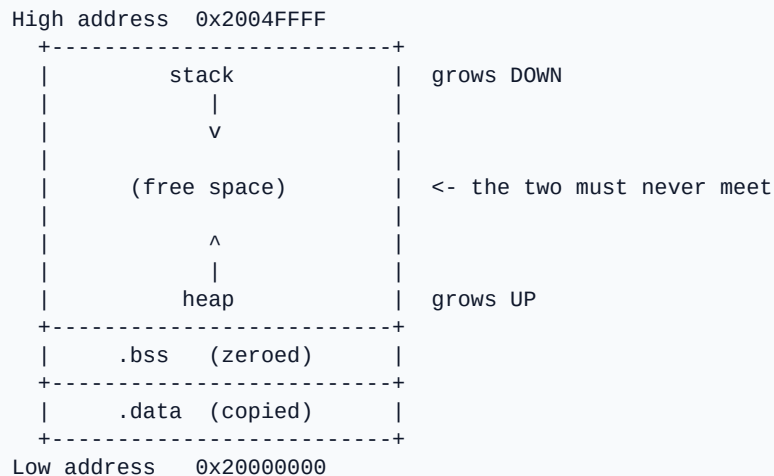
    /* 2. Zero .bss */
    dst = &_sbss;
    while (dst < &_ebss) *dst++ = 0;

    /* 3. Now C guarantees hold, so main can run */
    main();
    for (;;) { }                    /* main must never return */
}
```

This is why globals are zero-initialised

The C standard guarantees that a static or global variable with no initialiser is zero. On a hosted system the OS provides zeroed pages. On bare metal, that guarantee is literally the .bss-zeroing loop above. If someone removes it from the startup file to save a few microseconds of boot time, every uninitialised global silently contains whatever was in SRAM from the previous run.

19.4 Stack and Heap Collision



Collision is silent

When the stack grows into the heap, or the heap into the stack, nothing announces it. There is no MMU and no guard page. Local variables start overwriting heap blocks, or malloc returns memory that the stack is already using. The symptom is corrupted data in an unrelated part of the program, appearing minutes later. This is among the hardest classes of embedded bug to diagnose, and it is the main argument for avoiding the heap altogether.

19.5 Measuring What You Are Using

```
$ arm-none-eabi-size -A firmware.elf

section      size      addr
.text        48216    0x80000000    <- Flash: code
.rodata       3104    0x800BC58      <- Flash: constants
.data         512     0x20000000    <- SRAM (and a Flash copy)
.bss        198432    0x20000200    <- SRAM: the big one

Flash used  = .text + .rodata + .data = 51,832 of 1,048,576  (5%)
SRAM used   = .data + .bss           = 198,944 of 327,680  (61%)
Left for stack + heap = 128,736 bytes
```

Where ODT's SRAM actually goes

For a small HAR classifier the accounting looks roughly like this. Weights in float32: perhaps 40 KB. Gradients, same shape: another 40 KB. AdamW keeps a first and second moment per parameter: 80 KB more. Activation buffers for the forward pass: 20 KB. The RgL sparsity masks, packed one bit per weight: 1.25 KB — almost free by comparison. The optimiser state is the single largest consumer, which is precisely why plain SGD, with no moments at all, remains a serious option on this hardware even though AdamW converges faster.

19.6 Placing Data Deliberately

```
/* GCC section attribute - put a buffer in the fast DTCM */
__attribute__((section(".dtcm")))
static float hotBuffer[1024];

/* Force 32-byte alignment for cache-line-friendly DMA */
__attribute__((aligned(32)))
static uint8_t dmaBuffer[512];

/* Keep a symbol the linker would otherwise discard */
__attribute__((used))
static const char version[] = "ODT 1.0";
```

The section name must exist in the linker script. Moving the hottest inner-loop buffer into DTCM removes wait states entirely and can be worth 10 to 20 percent on a memory-bound kernel such as a matmul.

Exercises — Chapter 19

1. Classify each of these into a section: a const lookup table, an initialised global, an uninitialised static array, a function local, a malloc result.
2. Compute the SRAM cost of `int buf[5000];` at file scope.
3. Explain why marking that array `const` changes its cost, and by how much.
4. Run `size` on any compiled binary and identify `.text`, `.data` and `.bss`.
5. A model has 12000 float32 parameters. Compute total SRAM for weights, gradients and AdamW moments. Does it fit in 320 KB?
6. Compute the packed mask size for those 12000 weights.
7. Explain what happens, step by step, if the `.bss`-zeroing loop is removed from the startup code.
8. Why must `main` never return on an embedded target?

volatile, const and restrict

Type qualifiers that change what the compiler may assume

20.1 volatile: This May Change Behind Your Back

The compiler normally assumes that memory only changes when the program writes to it. That assumption is false for hardware registers, for variables modified by an interrupt handler, and for memory shared with a DMA controller. `volatile` withdraws the assumption.

```
/* WITHOUT volatile: the compiler may cache flag in a register,
   never re-read it, and turn this into an infinite loop. */
bool flag = false;
while (!flag) { }                /* may never exit, even after the ISR sets flag */

/* WITH volatile: forces a fresh load from memory every iteration. */
volatile bool flag = false;
while (!flag) { }                /* correct */

void EXTI0_IRQHandler(void) { /* interrupt handler */
    flag = true;
}
```

Use volatile for	Reason
Memory-mapped hardware registers	Reading has side effects; the value changes independently of the CPU
Variables written by an ISR	The main loop cannot see the write otherwise
DMA buffers	The DMA engine writes without the CPU knowing
Variables in a <code>setjmp/longjmp</code> path	Otherwise their values after <code>longjmp</code> are indeterminate

volatile is not atomicity and not a memory barrier

It guarantees that each access happens and is not reordered relative to other volatile accesses. It does not make a read-modify-write atomic. `volatile int c; c++;` is still three separate operations and an interrupt between them loses an increment. For atomicity use `<stdatomic.h>` or disable interrupts around the critical section.

```
/* Hardware register access - the standard CMSIS pattern */
#define GPIOB_ODR (*(volatile uint32_t *)0x40020414)

GPIOB_ODR |= (1u << 7);          /* set pin 7 */
GPIOB_ODR &= ~(1u << 7);         /* clear pin 7 */

/* Without volatile the compiler could merge or delete these writes,
   because from its point of view nothing reads the location. */
```

20.2 const: I Will Not Modify This

```
const int x = 5;           /* an object that cannot be modified */

const int *p;             /* pointer to const int:  *p = 1 is an error */
int const *p;             /* identical meaning */
int * const p;            /* const pointer:  p = &y is an error */
const int * const p;      /* neither can change */
```

Read declarations right to left, and remember that `const` applies to whatever is immediately to its left — or, if nothing is to its left, to whatever is on its right.

Benefit	Detail
Compile-time safety	Accidental writes become errors instead of bugs
Documentation	The signature tells the caller what the function does to its arguments
Flash placement	<code>const</code> file-scope data goes to <code>.rodata</code> , saving SRAM
Optimisation	The compiler may cache the value in a register across calls

const-correctness in the ODT tensor API

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex);
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value);
```

You can tell from the signatures alone which one writes. The getter promises not to touch the tensor, so a caller holding a `const` tensor can still read it. If someone later adds a stray write inside the getter, the build fails rather than silently corrupting data.

const in C is weaker than in C++

A `const int` at file scope in C is not a compile-time constant expression — you cannot use it as an array size or a case label. Use `#define` or an enum for those. C++ made `const` stronger; C did not.

20.3 restrict: These Pointers Do Not Overlap

```
/* Without restrict, the compiler must assume a and b may alias,
   so it must reload b[i] after every write to a[i]. */
void addArrays(float *a, const float *b, size_t n) {
    for (size_t i = 0; i < n; i++) a[i] += b[i];
}

/* With restrict, you promise they do not overlap.
   The compiler can now vectorise and keep values in registers. */
void addArrays(float * restrict a, const float * restrict b, size_t n) {
    for (size_t i = 0; i < n; i++) a[i] += b[i];
}
```

restrict is a promise you must keep

If the pointers do overlap after all, the behaviour is undefined and the generated code will produce wrong results. There is no diagnostic. Use `restrict` only where you control every caller and can guarantee separation. This is exactly why `memcpy` is declared with `restrict` and `memmove` is not.

20.4 Combining Qualifiers

```

/* A read-only hardware register: the hardware changes it,
   but the program must never write it. */
const volatile uint32_t * const statusReg = (uint32_t *)0x40020410;

uint32_t s = *statusReg;      /* fine - fresh read every time */
/* *statusReg = 1;           error - const */
/* statusReg = otherAddr;    error - const pointer */

```

`const volatile` is not a contradiction. `const` constrains the program; `volatile` describes the world. Together they say: something else changes this, and you may only read it.

Exercises — Chapter 20

1. Write a busy-wait loop on a flag set by an ISR. Show what breaks without `volatile`.
2. Explain the difference between `const int *p`, `int * const p` and `const int * const p`.
3. Take a function with three pointer parameters and add `const` wherever it is correct.
4. Explain why `volatile int c; c++;` is not interrupt-safe.
5. Define a macro for a memory-mapped 32-bit register at `0x40021000` and write set/clear operations on bit 3.
6. When is `restrict` unsafe? Give a concrete failing example.
7. Explain why `const volatile` is meaningful rather than contradictory.

Programming Without malloc

Static pools, arenas, and predictable memory

21.1 Why Ban the Heap

Problem	Consequence on an MCU
Fragmentation	A large allocation fails while plenty of total memory is free
Non-determinism	malloc has no bounded execution time; unacceptable in a real-time loop
Failure handling	Every call site must handle NULL, and most code does not
Invisible exhaustion	No MMU, no guard page — overflow silently corrupts the stack
Code size	A general-purpose allocator is several kilobytes of Flash

Safety-critical standards go further: MISRA C, the JPL coding standard and most automotive guidelines ban dynamic allocation after initialisation outright. If the memory requirement cannot be computed at build time, it cannot be certified.

21.2 Static Allocation

```
/* Size known at compile time - the linker reserves it in .bss */
#define MAX_WEIGHTS 16384
static float weights[MAX_WEIGHTS];
static float grads [MAX_WEIGHTS];

/* Packed sparsity mask: one bit per weight */
static uint8_t mask[(MAX_WEIGHTS + 7) / 8];

/* Nothing can fail at runtime. If it does not fit, the LINK fails. */
```

A link error is the best kind of error

When a static array does not fit, the linker says so before the firmware is ever flashed. A malloc that fails says so only in the field, in a code path nobody tested. Moving memory failures from runtime to build time is the whole point of static allocation.

21.3 Arena (Bump) Allocation

When you need variable-sized allocations but not variable lifetimes, an arena is ideal: allocate by advancing a pointer, and free everything at once by resetting it.

```

typedef struct {
    uint8_t *base;
    size_t   size;
    size_t   used;
} arena_t;

void *arenaAlloc(arena_t *a, size_t bytes, size_t align) {
    /* Round the current offset up to the requested alignment */
    size_t off = (a->used + align - 1) & ~(align - 1);
    if (off + bytes > a->size) return NULL;    /* out of arena */
    a->used = off + bytes;
    return a->base + off;
}

void arenaReset(arena_t *a) {
    a->used = 0;    /* frees everything in O(1) */
}

/* Backing store is a plain static array */
static uint8_t arenaBuffer[64 * 1024];
static arena_t arena = { arenaBuffer, sizeof(arenaBuffer), 0 };

```

Property	Arena	malloc
Allocation cost	A few instructions	Search a free list; unbounded
Free cost	One store, and only for everything at once	Coalescing; unbounded
Fragmentation	Impossible	Inevitable over time
Individual free	Not supported	Supported
Metadata overhead	None	8 to 16 bytes per block

The alignment expression $(x + a - 1) \& \sim(a - 1)$ rounds x up to the next multiple of a , and works only when a is a power of two — which every natural alignment is.

21.4 Fixed-Size Pools

When every allocation is the same size, a free list of fixed blocks gives $O(1)$ allocate and $O(1)$ free with zero fragmentation.

```

#define POOL_BLOCKS 32
#define BLOCK_SIZE 1152    /* one HAR window of floats */

static float  poolMem[POOL_BLOCKS][BLOCK_SIZE];
static bool   poolUsed[POOL_BLOCKS];

float *poolAlloc(void) {
    for (size_t i = 0; i < POOL_BLOCKS; i++) {
        if (!poolUsed[i]) { poolUsed[i] = true; return poolMem[i]; }
    }
    return NULL;    /* pool exhausted */
}

void poolFree(float *p) {
    size_t i = (size_t)(p - &poolMem[0][0]) / BLOCK_SIZE;
    if (i < POOL_BLOCKS) poolUsed[i] = false;
}

```

The poolUsed array can itself be bit-packed, at which point finding a free block becomes a scan for a clear bit — the same technique as the RigL mask, applied to memory management.

21.5 Caller-Allocates

The cleanest discipline of all is to have functions never allocate. The caller supplies every buffer, so ownership is unambiguous and there is nothing to leak.

```
/* Function allocates - caller must remember to free */
tensor_t *makeTensor(size_t n);          /* who frees this? */

/* Caller allocates - no ambiguity, no leak possible */
void initTensor(tensor_t *t, uint8_t *storage, size_t n);

/* Usage */
static uint8_t backing[4608];
tensor_t window;
initTensor(&window, backing, 1152);
```

This is ODT's rule for the training loop

Weights, gradients, AdamW moments, activation buffers and the sparsity masks are all allocated once during setup. From the first training step onward, not one call to malloc occurs. Layer functions such as linearForward(layer, input, output) receive the output tensor rather than returning one. The result is a training loop whose memory footprint is constant and knowable from the linker map, which is exactly what you need when there is no way to observe a fragmentation failure until the device stops working.

21.6 alloca and Variable-Length Arrays

```
#include <alloca.h>
void f(size_t n) {
    float *scratch = alloca(n * sizeof(float)); /* on the STACK */
    /* automatically released when f returns - no free needed */
}

/* C99 VLA - similar effect, standard syntax */
void g(size_t n) {
    float scratch[n];
}
```

Both can blow the stack with no warning

If n comes from data, a large value silently overruns the stack. There is no failure return from alloca — it cannot report failure. Use them only with a hard bound that you check first. ODT uses stack scratch inside executeOp, but the sizes are derived from tensor shapes that were validated at model construction time.

Exercises — Chapter 21

1. Implement an arena allocator with alignment and test it with mixed-size allocations.
2. Explain why $(x + a - 1) \& \sim(a - 1)$ rounds up, and why it requires a to be a power of two.
3. Implement a fixed-size pool using a bit-packed used-flag array.
4. Rewrite a function that returns a malloc'd buffer into caller-allocates form.
5. Compute the total static SRAM for a model with 8000 weights: weights, gradients, two AdamW moments, and a packed mask.
6. Explain how a link error is preferable to a runtime malloc failure.
7. Give a concrete scenario where a `alloca` overflows the stack, and how to prevent it.

Chapter 22

Function Pointers and Dispatch Tables

Runtime polymorphism without a vtable

22.1 Declaring a Function Pointer

```
/* p points to a function taking (int, int) and returning int */
int (*p)(int, int);

int add(int a, int b) { return a + b; }

p = add;          /* the & is optional - a function name decays to a pointer */
int r = p(2, 3);   /* the * is optional too - both forms call it */
int r = (*p)(2, 3);

/* A typedef makes this readable */
typedef int (*binop_t)(int, int);
binop_t op = add;
```

The parentheses around *p are mandatory

`int *p(int, int);` declares a *function returning int*. `int (*p)(int, int);` declares a *pointer to a function returning int*. The parentheses change the meaning entirely, because `( )` binds tighter than `*`.

22.2 Dispatch Tables

The ODT layer dispatch table

```
typedef enum { LINEAR, CONV1D, RELU, SOFTMAX, LAYER_COUNT } layerType_t;

typedef struct {
    void (*forward) (layer_t *, tensor_t *, tensor_t *);
    void (*backward)(layer_t *, tensor_t *, tensor_t *);
    void (*calcOutputShape)(layer_t *, shape_t *, shape_t *);
} layerOps_t;

/* C99 designated initialisers: sparse, order-independent, self-documenting */
static const layerOps_t layerFunctions[LAYER_COUNT] = {
    [LINEAR] = { linearForward,  linearBackward,  linearCalcOutputShape },
    [CONV1D] = { conv1dForward,  conv1dBackward,  conv1dCalcOutputShape },
    [RELU]    = { reluForward,   reluBackward,   reluCalcOutputShape },
    [SOFTMAX] = { softmaxForward, NULL,          softmaxCalcOutputShape },
};

/* Dispatch is one indexed load and one indirect call */
void layerForward(layer_t *l, tensor_t *in, tensor_t *out) {
    layerFunctions[l->type].forward(l, in, out);
}
```

Property	Dispatch table	switch statement
Dispatch cost	$O(1)$ — indexed load plus indirect call	$O(1)$ with a jump table, $O(n)$ if the compiler emits comparisons

Property	Dispatch table	switch statement
Adding a type	One new table row	Edit every switch in the codebase
Storage	Table lives in Flash as const	Code in Flash
Missing case	NULL pointer, checkable at runtime	Falls to default, or silently does nothing
Readability	All behaviour for a type on one line	Behaviour scattered across several switches

NULL means not supported

Notice [SOFTMAX] = { softmaxForward, NULL, ... }. A NULL backward pointer encodes "this layer is inference-only", and the training loop simply checks before calling. Unspecified entries in a designated initialiser are zero-initialised, so a layer type someone forgot to register gets an all-NULL row rather than a garbage function pointer — a crash you can diagnose instead of one you cannot.

22.3 Callbacks with Context

ODT opSpec_t

```
typedef void (*kernel_t)(void *ctx, const tensor_t **in, size_t nIn, tensor_t *out);

typedef struct {
    kernel_t      kernel;
    void          *ctx;           /* opaque, kernel-specific parameters */
    const tensor_t *inputs[4];
    size_t        nInputs;
    arithmetic_t   arithmetic;
    outputMode_t   mode;
    bool           writesInPlaceSafe;
} opSpec_t;

/* Each kernel casts ctx back to its own parameter struct */
static void matmulKernel(void *ctx, const tensor_t **in, size_t nIn, tensor_t *out) {
    matmulCtx_t *p = (matmulCtx_t *)ctx;
    /* ... use p->transposeA, p->alpha, etc ... */
}
```

This is C's equivalent of a closure. The void * carries whatever state the callback needs, and only the callback knows its real type. One executeOp function can then drive matmul, elementwise add, and reduction kernels without knowing anything about their individual parameters.

22.4 Sorting with a Comparator

```
void qsort(void *base, size_t n, size_t size,
           int (*compare)(const void *, const void *));

/* Sort weight indices by descending gradient magnitude - the RgL grow step */
static const float *g_grads; /* comparator context, via a file-scope pointer */

static int cmpByGradDesc(const void *a, const void *b) {
    size_t ia = *(const size_t *)a;
    size_t ib = *(const size_t *)b;
    float ga = fabsf(g_grads[ia]);
    float gb = fabsf(g_grads[ib]);
    return (ga < gb) - (ga > gb); /* descending */
}
```

qsort comparators cannot take context

The standard signature has no context parameter, which forces the file-scope global above — not reentrant, and unusable from two threads. POSIX offers `qsort_r` and C11 offers `qsort_s`, both of which take a context pointer, but neither is portable. This limitation is one more reason ODT hand-writes its partial selection sort for `RigL` rather than calling `qsort`.

22.5 State Machines

```
typedef enum { ST_IDLE, ST_COLLECT, ST_TRAIN, ST_SAVE, ST_COUNT } state_t;

typedef state_t (*stateFn_t)(context_t *);

static const stateFn_t stateTable[ST_COUNT] = {
    [ST_IDLE]     = onIdle,
    [ST_COLLECT]  = onCollect,
    [ST_TRAIN]    = onTrain,
    [ST_SAVE]     = onSave,
};

for (;;) {
    state = stateTable[state](&ctx);    /* each handler returns the next state */
}
```

Each handler is a small, independently testable function, and the transition logic lives inside the handlers rather than in one enormous switch. This structure scales to dozens of states without becoming unreadable.

Exercises — Chapter 22

1. Declare a pointer to a function taking two floats and returning a float; assign it and call it.
2. Explain the difference between `int *f(void)` and `int (*f)(void)`.
3. Build a dispatch table of four arithmetic operations indexed by an enum.
4. Write a `qsort` comparator that sorts a struct array by one float field.
5. Explain why an unspecified designated-initialiser entry is safer than an uninitialised one.
6. Implement a three-state machine using a function pointer table.
7. Write a generic `forEach(void *arr, size_t n, size_t sz, void (*fn)(void *, void *), void *ctx)`.
8. Explain what `void *ctx` in `opSpec_t` buys you over adding parameters to the kernel signature.

Alignment, Padding and Endianness

How the machine really lays out your data

23.1 Alignment

Most processors require a multi-byte value to sit at an address that is a multiple of its size. A 4-byte int must start at an address divisible by 4. This is called *natural alignment*, and violating it either costs extra cycles or faults outright depending on the instruction and the core.

Type	Size	Required alignment
char	1	1 — any address
short	2	2
int / float	4	4
double	8	8 (4 on some 32-bit ABIs)
pointer	4 or 8	Same as its size
struct	varies	Alignment of its most-aligned member

```
#include <stdalign.h>
printf("%zu\n", alignof(float));    /* 4 */

/* Force stricter alignment - useful for DMA and cache lines */
alignas(32) static uint8_t dmaBuffer[512];
__attribute__((aligned(32))) static uint8_t buf2[512];    /* GCC equivalent */
```

23.2 Struct Padding

```
struct bad {
    char a;    /* offset 0 */
               /* 3 bytes padding so b lands on a multiple of 4 */
    int b;    /* offset 4 */
    char c;    /* offset 8 */
               /* 3 bytes tail padding so the struct size is a multiple of 4 */
};    /* sizeof = 12; members total only 6 */

struct good {
    int b;    /* offset 0 */
    char a;    /* offset 4 */
    char c;    /* offset 5 */
               /* 2 bytes tail padding */
};    /* sizeof = 8 */
```

Tail padding exists so that an array of the struct keeps every element aligned. Ordering members from largest to smallest minimises the total. For a struct instantiated thousands of times — a tensor descriptor, a layer config — the saving is real.


```
#include <stddef.h>
printf("%zu\n", offsetof(struct bad, b)); /* 4 */
printf("%zu\n", sizeof(struct bad));    /* 12 */
```

Never memcmp two structs

The padding bytes are uninitialised and hold whatever was previously in that memory. Two structs with identical member values can compare unequal. Compare member by member, or zero the whole struct with `memset` before filling it if you genuinely need byte comparison.

Never fwrite a struct to a file you intend to read elsewhere

The layout depends on the compiler, the target architecture and the optimisation settings. A struct written by one build may not be readable by another. Serialise field by field with explicit sizes and byte order — Chapter 26.

23.3 Packed Structs

```
/* Removes all padding - matches an external binary layout exactly */
struct __attribute__((packed)) header {
    uint8_t magic[4];
    uint32_t version;
    uint32_t layerCount;
}; /* sizeof = 12 exactly, with no padding */
```

Packed members may be unaligned

Taking the address of a packed member gives a possibly-unaligned pointer. Dereferencing it on ARM can raise a `UsageFault`, and passing it to a function expecting an aligned pointer is undefined behaviour. Packed structs are for describing external formats, not for saving RAM in your own data structures. Prefer explicit `memcpy` field-by-field parsing.

23.4 Endianness

Endianness is the order in which the bytes of a multi-byte value are stored. It affects nothing while data stays inside one machine, and everything the moment data crosses a boundary — a file, a network, a peripheral.

```
uint32_t v = 0x12345678;

Little-endian (x86, ARM Cortex-M by default):
address: +0 +1 +2 +3
byte:    78 56 34 12      <- least significant byte first

Big-endian (network byte order, some MIPS/PowerPC):
address: +0 +1 +2 +3
byte:    12 34 56 78      <- most significant byte first
```

```

/* Detect at runtime */
static bool isLittleEndian(void) {
    uint16_t x = 1;
    return *(uint8_t *)&x == 1;
}

/* Read a little-endian uint32 from a byte buffer, portably.
   This works identically on a big-endian host. */
static uint32_t readLE32(const uint8_t *p) {
    return (uint32_t)p[0]
        | ((uint32_t)p[1] << 8)
        | ((uint32_t)p[2] << 16)
        | ((uint32_t)p[3] << 24);
}

static void writeLE32(uint8_t *p, uint32_t v) {
    p[0] = (uint8_t)(v);
    p[1] = (uint8_t)(v >> 8);
    p[2] = (uint8_t)(v >> 16);
    p[3] = (uint8_t)(v >> 24);
}

```

Byte-at-a-time access is always portable

The `readLE32` function above produces the correct value on any host, big- or little-endian, because it never reinterprets memory — it builds the number arithmetically from individual bytes. This costs a handful of instructions and removes an entire category of portability bug. Use it for every binary format.

Endianness in the .npy and ODTs formats

The NumPy header dict carries the byte order in its dtype string: `'<f4'` means little-endian float32, `'>f4'` big-endian. ODT parses the `<` and rejects anything else. Since both x86 and Cortex-M are little-endian, no swapping is ever needed in practice — but the check is there so that a big-endian file fails loudly rather than producing silently wrong training data. ODT's own ODTs checkpoint format is documented as little-endian for the same reason: an undocumented byte order is a bug waiting for a different machine.

Exercises — Chapter 23

1. Compute `sizeof` and the offset of each member for `struct { char a; double b; char c; short d; }`.
2. Reorder those members to minimise the size. How many bytes do you save?
3. Write a program that prints the four bytes of `0x12345678` in memory order and states the host endianness.
4. Implement `readLE32` and `writeLE32` and verify they round-trip.
5. Explain why `memcmp` on two structs can report a difference even when all members are equal.
6. Declare a packed struct matching the 12-byte ODTs header and confirm its size.
7. Explain the risk of taking the address of a packed struct member on ARM.
8. Write `readBE32` for big-endian input and test it against known bytes.

Chapter 24

Overflow and Undefined Behaviour

The traps the compiler is allowed to exploit

24.1 What Undefined Behaviour Means

Undefined behaviour is not "the result is unpredictable". It is stronger: the standard places **no requirements at all** on the program. The compiler is entitled to assume UB never happens and to optimise on that assumption. In practice this means code around the UB can be deleted, reordered, or replaced with something that makes no sense — and the behaviour frequently changes between -O0 and -O2.

```
/* The compiler assumes signed overflow cannot happen. */
int f(int x) {
    if (x + 1 < x) return -1;    /* would only be true on overflow */
    return 0;
}
/* At -O2, GCC deletes the if entirely and compiles this to "return 0". */
```

24.2 Signed vs Unsigned Overflow

Case	Behaviour
Unsigned overflow	Well defined. Wraps modulo 2^N . <code>UINT32_MAX + 1 == 0</code> .
Signed overflow	Undefined. May wrap, may saturate, may cause the compiler to delete surrounding code.

```
uint32_t u = UINT32_MAX;
u = u + 1;                /* defined: u becomes 0 */

int32_t s = INT32_MAX;
s = s + 1;                /* UNDEFINED - anything may happen */

/* Safe signed addition: check BEFORE adding */
bool addSafe(int32_t a, int32_t b, int32_t *out) {
    if (b > 0 && a > INT32_MAX - b) return false;    /* would overflow */
    if (b < 0 && a < INT32_MIN - b) return false;    /* would underflow */
    *out = a + b;
    return true;
}

/* Or use the builtin, which the hardware can do in one instruction */
if (__builtin_add_overflow(a, b, &result)) { /* overflowed */ }
```

24.3 The Common UB List

Undefined behaviour	Example
Signed integer overflow	<code>INT_MAX + 1</code>
Division by zero	<code>x / 0, x % 0</code>
Shift by $\geq$ width	<code>1u << 32</code> on a 32-bit type

Undefined behaviour	Example
Shift by a negative amount	<code>x << -1</code>
Out-of-bounds array access	<code>arr[n]</code> for an n-element array
Dereferencing NULL	<code>*p</code> when <code>p</code> is NULL
Use after free	Reading memory already passed to free
Reading an uninitialised variable	<code>int x; printf("%d", x);</code>
Modifying an object twice in one expression	<code>i = i++ + 1</code>
Strict-aliasing violation	Reading a float through an <code>int*</code> cast
Casting an out-of-range float to int	<code>(int8_t)1000.0f</code>
Modifying a string literal	<code>char *p = "abc"; p[0] = 'X';</code>

24.4 Strict Aliasing

```
/* UB: accessing a float through an int pointer */
float f = 3.14f;
int *p = (int *)&f;
int i = *p; /* strict-aliasing violation */

/* Correct and just as fast - the optimiser reduces this to a move */
float f = 3.14f;
int i;
memcpy(&i, &f, sizeof(i));

/* Also acceptable in practice: a union */
union { float f; int i; } u = { .f = 3.14f };
int i = u.i;
```

The rule is that an object may only be accessed through a compatible type, or through `char *`. Violating it lets the compiler assume the float and the int cannot refer to the same memory, and reorder loads and stores accordingly. The bug appears only at high optimisation levels, which is what makes it so unpleasant.

24.5 Finding UB

```
# UndefinedBehaviorSanitizer - traps signed overflow, bad shifts, misalignment
gcc -fsanitize=undefined -g prog.c -o prog

# AddressSanitizer - out-of-bounds, use-after-free, leaks
gcc -fsanitize=address -g prog.c -o prog

# Both together
gcc -fsanitize=address,undefined -g -O1 prog.c -o prog

# Useful warnings
gcc -Wall -Wextra -Wconversion -Wsign-conversion -Wshadow -Wstrict-aliasing=2
```

Run the sanitisers on your host build

ODT compiles for both the STM32 and the development PC. The host build exists partly so that the sanitisers can run on it — they are not available on bare metal. Any bug the sanitiser finds on the host is a bug that would have been invisible on the device, where an out-of-bounds write simply corrupts a neighbouring tensor and shows up as an inexplicable accuracy drop three epochs later.

Exercises — Chapter 24

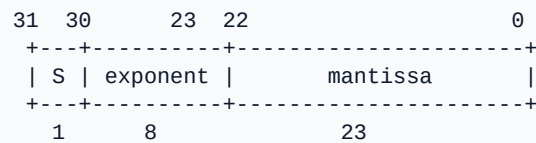
1. Write a program that overflows a signed int. Compile at `-O0` and `-O2` and compare the output.
2. Do the same with an unsigned int and explain why the result is stable.
3. Write a safe signed addition function with overflow checking.
4. Explain why the compiler may delete `if (x + 1 < x)`.
5. Demonstrate a strict-aliasing violation, then fix it with `memcpy`.
6. Compile a program with an out-of-bounds write under `-fsanitize=address` and read the report.
7. List five undefined behaviours you might plausibly write by accident, and the defence for each.
8. Explain why `(int8_t)1000.0f` is UB and what the correct conversion looks like.

Chapter 25

Floating Point on Microcontrollers

IEEE 754, the FPU, and when to avoid both

25.1 IEEE 754 float32 Layout



$\text{value} = (-1)^S \times 1.\text{mantissa} \times 2^{(\text{exponent} - 127)}$

```
/* 1.0f = 0x3F800000: S=0, exp=127, mantissa=0 -> 1.0 x 2^0 */
```

The leading 1 in "1.mantissa" is implicit and not stored, which buys an extra bit of precision. That gives 24 effective mantissa bits, or about 7.2 decimal digits.

Encoding	Meaning
exp = 0, mantissa = 0	Zero (positive or negative)
exp = 0, mantissa != 0	Subnormal — gradual underflow, often very slow
exp = 255, mantissa = 0	Infinity
exp = 255, mantissa != 0	NaN (Not a Number)

25.2 Precision and Spacing

Magnitude	Gap to the next float
1.0	1.19e-7
1000.0	6.1e-5
1000000.0	0.0625
16777216.0 (2 ²⁴)	2.0

```
/* Beyond 2^24, consecutive integers are not all representable */
float f = 16777216.0f;      /* 2^24 */
f + 1.0f == f;              /* TRUE - the increment is lost */

/* Accumulating many small values loses the small ones */
float sum = 0.0f;
for (int i = 0; i < 10000000; i++) sum += 0.1f;
/* Expected 1000000.0, actual is noticeably different */
```

Kahan summation recovers the lost bits

Keep a running compensation term holding the error from each addition and add it back on the next iteration. It costs three extra operations per element and makes a float32 sum nearly as accurate as a double sum. Worth it whenever you are averaging thousands of values, such as computing a loss over a large batch.

```
float kahanSum(const float *a, size_t n) {
    float sum = 0.0f, c = 0.0f;
    for (size_t i = 0; i < n; i++) {
        float y = a[i] - c;          /* apply the running compensation */
        float t = sum + y;
        c = (t - sum) - y;           /* capture what was lost this time */
        sum = t;
    }
    return sum;
}
```

25.3 Comparing Floats

```
/* WRONG */
if (a == b) { }
```

```
/* Absolute tolerance - fine when the magnitudes are known and near 1 */
if (fabsf(a - b) < 1e-6f) { }
```

```
/* Relative tolerance - correct across magnitudes */
bool nearlyEqual(float a, float b, float relTol) {
    float diff = fabsf(a - b);
    float scale = fmaxf(fabsf(a), fabsf(b));
    return diff <= relTol * scale || diff < FLT_MIN;
}
```

```
/* NaN compares false against everything, including itself */
if (x != x) { /* x is NaN */ }
if (isnan(x)) { /* clearer - from math.h */ }
```

25.4 The Cortex-M7 FPU

Operation	Cycles (approximate)
float add / subtract / multiply	1
float multiply-accumulate (VFMA)	1
float divide	14
float square root	14
expf / logf / sinf (library)	30 to 100
double add or multiply	Software emulation — tens of cycles

The FPU is single precision only

The Cortex-M7 FPU handles float32 in hardware. Every double-precision operation is emulated in software and can be an order of magnitude slower. A single forgotten `f` suffix — `x * 2.0` instead of `x * 2.0f` — promotes the whole expression to double and can dominate the runtime of an inner loop.

```

/* Enable the FPU before using it - the compiler will not do this for you */
SCB->CPACR |= (0xFu << 20); /* full access to CP10 and CP11 */
__DSB(); __ISB(); /* ensure the write lands before the next instruction */

/* Compiler flags for hardware float */
-mcpu=cortex-m7 -mfpu=fpv5-sp-d16 -mfloat-abi=hard

```

Where ODT deliberately pays for double

ODT is float32 throughout, with exactly one exception: the AdamW hyperparameters `beta1`, `beta2`, `eps` and `weightDecay` are declared `double`. The reason is the bias-correction term. `1.0f - 0.999f` in float32 loses three decimal places to catastrophic cancellation, and that error is then raised to the power of the step count thousands of times. These four values are scalars, computed once per step, so the software-emulation cost is negligible — while the precision they buy is the difference between a stable optimiser and a drifting one. Everything inside the per-element kernels stays float32.

25.5 Avoiding Float Entirely

Technique	When it applies
Fixed-point (Chapter 18)	The range is known and bounded
Integer-only algorithms	Counters, indices, comparisons
Lookup tables in Flash	Transcendental functions over a small domain
Polynomial approximation	exp, log, sin over a restricted range
Reciprocal multiply	Replace repeated division by a precomputed $1/x$

```

/* Division is 14 cycles; multiplication is 1. Hoist the reciprocal. */
float inv = 1.0f / scale; /* one divide, outside the loop */
for (size_t i = 0; i < n; i++) {
    q[i] = roundf(x[i] * inv); /* n multiplies instead of n divides */
}
/* For n = 1152 this saves roughly 15000 cycles per tensor. */

```

This exact hoist appears in ODT's `float32ToSym` quantisation function, which computes `float inv = 1.0f / q->scale`; once before the element loop.

Exercises — Chapter 25

1. Print the raw bits of `1.0f`, `0.5f` and `-2.0f` as hex and decode the sign, exponent and mantissa of each.
2. Show that `0.1f + 0.2f != 0.3f` and print the actual difference.
3. Sum `0.1f` ten million times naively and with Kahan summation. Compare both to the exact answer.
4. Write a relative-tolerance float comparison and test it near `1.0` and near `1e6`.
5. Find the smallest float `x` for which `x + 1.0f == x`.
6. Time a loop of 100000 divisions against the same loop with a hoisted reciprocal.
7. Explain why ODT stores AdamW `beta2` as `double` but the weights as `float`.
8. Write an `isnan` check without using `math.h`.

Binary Formats and Serialisation

Writing data that another program can read

26.1 Design Rules

Rule	Why
Magic bytes first	Reject the wrong file type immediately, before parsing anything
Version field next	Old readers can refuse new files with a clear message
Fixed-width types only	int is 16 bits somewhere; uint32_t is 32 bits everywhere
Document the byte order	Little-endian, and enforce it on read
Length-prefix variable data	The reader knows how much to consume without scanning
Never fwrte a struct	Padding and layout differ between compilers and targets

26.2 The ODTs Checkpoint Format

Offset	Size	Field
0	4	magic: 'O' 'D' 'T' 'S'
4	4	version: uint32 = 2 (little-endian)
8	4	layerCount: uint32
12	...	per-layer records, each:
		1 byte tag (layer type)
		N bytes payload (tag-dependent)

```
static bool writeHeader(FILE *f, uint32_t layerCount) {
    uint8_t hdr[12];
    memcpy(hdr, "ODTS", 4);
    writeLE32(hdr + 4, 2u);
    writeLE32(hdr + 8, layerCount);
    return fwrite(hdr, 1, 12, f) == 12;
}

static bool readHeader(FILE *f, uint32_t *outCount) {
    uint8_t hdr[12];
    if (fread(hdr, 1, 12, f) != 12)        return false;
    if (memcmp(hdr, "ODTS", 4) != 0)        return false; /* wrong format */
    uint32_t version = readLE32(hdr + 4);
    if (version != 2u)                      return false; /* wrong version */
    *outCount = readLE32(hdr + 8);
    return true;
}
```

Building the header in a byte array and writing it in one call is deliberate. It makes the on-disk layout explicit, it works identically on any host endianness, and it avoids the struct-padding problem entirely.

26.3 Tag-Length-Value Records

```

/* TLV lets a reader skip records it does not understand */
while (recordsRemaining--) {
    uint8_t tag;
    uint32_t length;
    if (fread(&tag, 1, 1, f) != 1) break;
    length = readLE32FromFile(f);

    switch (tag) {
        case TAG_LINEAR: readLinearLayer(f, length); break;
        case TAG_CONV1D: readConv1dLayer(f, length); break;
        default:
            fseek(f, (long)length, SEEK_CUR); /* skip unknown records */
            break;
    }
}

```

The length field is what makes forward compatibility possible. Without it, an unrecognised tag would leave the reader unable to find the start of the next record, and the whole file would have to be rejected.

26.4 Writing Tensors

```

static bool writeTensor(FILE *f, const tensor_t *t) {
    uint32_t nDims = (uint32_t)t->shape->numberOfDimensions;
    if (!writeLE32ToFile(f, nDims)) return false;
    for (uint32_t d = 0; d < nDims; d++) {
        if (!writeLE32ToFile(f, (uint32_t)t->shape->dimensions[d])) return false;
    }
    if (!writeLE32ToFile(f, (uint32_t)t->quantization->type)) return false;

    size_t bytes = calcBytesPerTensor(t);
    return fwrite(t->data, 1, bytes, f) == bytes;
}

```

serializeSparsity is an empty stub

ODT's `Serialize.c` declares and defines `serializeSparsity()`, but the body does nothing. The RIGL masks are therefore **not** written to the checkpoint. On restart the topology found by training is lost and the mask must be reinitialised from scratch. Filling in this stub is a concrete, self-contained task: write $(\text{numWeights} + 7) / 8$ bytes of packed mask per sparse layer under a new TLV tag, and the length field means older readers will skip it harmlessly.

26.5 Robust Reading

```

if (nDims == 0 || nDims > MAX_DIMS) return false; /* sanity-check first */
if (byteCount > MAX_TENSOR_BYTES) return false;
if (fread(buf, 1, byteCount, f) != byteCount) return false;

```

Validate every value read from a file

A corrupt or hostile file can contain a dimension count of 4 billion. If you use it to size a loop or an allocation without checking, you have a crash at best and an exploitable overflow at worst. Bound-check every length, count and index that comes from outside your program, before you use it for anything.

Exercises — Chapter 26

1. Write and read the 12-byte ODTs header, and verify the round trip.
2. Add a version check that rejects version 1 files with a clear error message.
3. Implement TLV writing and reading, including the skip path for unknown tags.
4. Write a function that serialises a float array with a length prefix.
5. Implement `serializeSparsity`: write a packed mask as a TLV record.
6. Implement the matching deserialiser and confirm a mask survives a save/load cycle.
7. Explain why writing a struct directly with `fwrite` is unsafe, with a concrete failure scenario.
8. List every field in your format that needs bound-checking on read, and the bound for each.

ARM Cortex-M Specifics

Registers, interrupts, cycle counting and CMSIS

27.1 The Cortex-M7 Core

Feature	Detail
Architecture	ARMv7-M, 32-bit, Thumb-2 instruction set
Clock	Up to 216 MHz on the STM32F746
Pipeline	6-stage, dual-issue, with branch prediction
Caches	4 KB instruction + 4 KB data (L1)
FPU	Single precision (FPv5-SP-D16)
DSP	SIMD instructions: SMLAD, SMUAD, QADD8, SSAT
MPU	Memory Protection Unit, 8 regions — no virtual memory

27.2 Memory-Mapped Registers

```

/* Peripherals appear as ordinary memory addresses */
#define PERIPH_BASE    0x40000000u
#define GPIOB_BASE     (PERIPH_BASE + 0x00020400u)

/* The CMSIS style: a volatile struct overlaid on the register block */
typedef struct {
    volatile uint32_t MODER;    /* offset 0x00 - pin mode */
    volatile uint32_t OTYPER;   /* offset 0x04 */
    volatile uint32_t OSPEEDR;  /* offset 0x08 */
    volatile uint32_t PUPDR;    /* offset 0x0C */
    volatile uint32_t IDR;       /* offset 0x10 - input data */
    volatile uint32_t ODR;       /* offset 0x14 - output data */
} GPIO_TypeDef;

#define GPIOB ((GPIO_TypeDef *)GPIOB_BASE)

GPIOB->ODR |= (1u << 7);    /* set pin PB7 */
GPIOB->ODR &= ~(1u << 7);   /* clear pin PB7 */

```

Every field is `volatile`, because the hardware changes them independently of the CPU and because reads can have side effects — some status registers clear a flag when read. Note also that the struct layout must exactly match the hardware offsets, which is why no padding may creep in; all fields being `uint32_t` guarantees that.

Read-modify-write on a register is not atomic

`GPIOB->ODR |= mask;` is a load, an OR and a store. An interrupt between the load and the store loses whatever the ISR wrote. Many STM32 peripherals provide a separate BSRR register for exactly this reason: writing a bit there sets or clears one pin atomically, with no read at all.

27.3 Interrupts

```
/* An ISR is an ordinary C function with a name the vector table expects */
void TIM2_IRQHandler(void) {
    if (TIM2->SR & TIM_SR_UIF) {
        TIM2->SR &= ~TIM_SR_UIF;    /* clear the flag, or it re-fires forever */
        tickCount++;                /* must be volatile */
    }
}

volatile uint32_t tickCount = 0;
```

ISR rule	Reason
Keep it short	Higher-priority work is blocked while it runs
No malloc	The allocator is not reentrant
No printf	Slow, not reentrant, and may block on the UART
Shared variables must be volatile	Otherwise the main loop caches a stale value
Clear the interrupt flag	Otherwise the handler is re-entered immediately
No floating point unless configured	FPU context save must be enabled for that ISR

```
/* Critical section: block interrupts around a non-atomic update */
uint32_t primask = __get_PRIMASK();
__disable_irq();
    sharedCounter++;                /* now safe */
__set_PRIMASK(primask);           /* restore - do NOT just call __enable_irq() */
```

Saving and restoring PRIMASK rather than unconditionally re-enabling matters because the code may itself be called from inside another critical section. Unconditionally enabling would silently break the outer one.

27.4 Cycle-Accurate Timing

```
/* The DWT unit contains a free-running cycle counter */
static void cycleCounterInit(void) {
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

uint32_t t0 = DWT->CYCCNT;
linearForward(layer, input, output);
uint32_t cycles = DWT->CYCCNT - t0;    /* wraps correctly: unsigned arithmetic */

float us = (float)cycles / 216.0f;    /* at 216 MHz */
```

The subtraction is correct even across a wrap

CYCCNT is a 32-bit counter that wraps every 19.9 seconds at 216 MHz. Because both values are uint32_t and unsigned overflow is defined to wrap modulo 2^{32} , $t1 - t0$ gives the right elapsed count even when the counter wrapped in between — provided the interval itself is under 19.9 seconds. This is one of the rare places where relying on wraparound is both correct and idiomatic.

27.5 Caches

```

SCB_EnableICache();
SCB_EnableDCache();

/* Before a DMA device READS a buffer the CPU wrote: */
SCB_CleanDCache_by_Addr((uint32_t *)buf, (int32_t)size);

/* Before the CPU READS a buffer DMA wrote: */
SCB_InvalidateDCache_by_Addr((uint32_t *)buf, (int32_t)size);

```

Cache coherency with DMA is manual

The Cortex-M7 data cache is not coherent with DMA. If the CPU wrote to a buffer and the data is still sitting in the cache, a DMA read fetches stale memory. If DMA wrote to memory and the CPU has a cached copy, the CPU reads stale data. You must clean before DMA reads and invalidate before CPU reads. Buffers must also be aligned to a 32-byte cache line, or the operations will affect neighbouring data.

27.6 DSP Instructions

```

#include "arm_math.h"

/* SMLAD: two 16x16 multiplies and a 32-bit accumulate, in one cycle */
int32_t acc = 0;
acc = __SMLAD(packedA, packedB, acc);

/* SSAT: saturating clamp to n bits, in one instruction */
int32_t clamped = __SSAT(value, 8); /* clamps to [-128, 127] */

/* CMSIS-DSP provides tuned kernels */
arm_dot_prod_f32(a, b, n, &result);
arm_mat_mult_f32(&matA, &matB, &matC);

```

A concrete optimisation opportunity in ODT

The int8 quantisation path currently clamps with a ternary: $r = r < -128 ? -128 : (r > 127 ? 127 : r)$; . That is two compares and two conditional moves. `__SSAT(r, 8)` does the same thing in a single instruction. Over a 524288-element weight tensor that is roughly 1.5 million instructions saved per quantisation pass. The matmul inner loop is a larger prize still: SMLAD processes two int16 products per cycle, which would roughly double throughput on the dominant kernel.

Exercises — Chapter 27

1. Write a macro for a 32-bit register at 0x40021000 and functions to set, clear and read bit 5.
2. Explain why every field in a CMSIS peripheral struct is volatile.
3. Write an ISR that increments a counter and clears its flag, and the main-loop code that reads it safely.
4. Explain why `reg |= mask` is unsafe in the presence of interrupts, and what BSRR solves.
5. Write a critical section that saves and restores PRIMASK, and explain why restoring beats unconditionally enabling.
6. Initialise DWT->CYCCNT and time a 1000-iteration loop.
7. Explain why the CYCCNT subtraction is correct across a counter wrap.
8. Describe the two cache maintenance operations needed around a DMA transfer, and when each applies.

Chapter 28

Debugging, Testing and Optimisation

Finding bugs and making code fast, in that order

28.1 Compiler Warnings Are Free Bug Reports

```
gcc -Wall -Wextra -Wconversion -Wsign-conversion -Wshadow \
    -Wcast-align -Wstrict-prototypes -Wmissing-prototypes \
    -Wpointer-arith -Wwrite-strings -std=c11 -O2 file.c

# Treat every warning as an error - the only sustainable policy
gcc -Werror ...
```

Warning	Catches
-Wall -Wextra	Unused variables, missing returns, sign comparison mismatches
-Wconversion	Implicit narrowing that silently loses data
-Wshadow	An inner variable hiding an outer one
-Wcast-align	Casts that increase alignment requirements
-Wmissing-prototypes	Non-static functions with no declaration in a header

28.2 gdb

```
gcc -g -O0 prog.c -o prog      # -g for symbols, -O0 so lines map to code
gdb ./prog

(gdb) break main                # or break file.c:42
(gdb) run
(gdb) next          / n        # step over
(gdb) step          / s        # step into
(gdb) finish        # run until this function returns
(gdb) print x        # print a variable
(gdb) print arr[3]@5 # print 5 elements starting at arr[3]
(gdb) x/16xb ptr     # examine 16 bytes in hex
(gdb) backtrace      / bt     # the call stack
(gdb) watch counter  # break when counter changes
(gdb) info locals
```

The @ syntax for printing array slices and watch for data breakpoints are the two commands most people never learn and most wish they had.

28.3 Unit Testing Without a Framework

```

#include <stdio.h>
#include <math.h>

static int testsRun = 0, testsFailed = 0;

#define CHECK(cond) do {
    testsRun++;
    if (!(cond)) {
        testsFailed++;
        printf("FAIL %s:%d %s\n", __FILE__, __LINE__, #cond);
    }
} while (0)

#define CHECK_NEAR(a, b, tol) CHECK(fabsf((a) - (b)) < (tol))

static void testBitPacking(void) {
    uint8_t data[2] = {0, 0};
    setBit(data, 15, true);
    CHECK(data[1] == 0x80); /* bit 7 of byte 1 */
    CHECK(getBit(data, 15) == true);
    CHECK(getBit(data, 14) == false);
    setBit(data, 15, false);
    CHECK(data[1] == 0x00);
}

int main(void) {
    testBitPacking();
    printf("%d/%d passed\n", testsRun - testsFailed, testsRun);
    return testsFailed != 0;
}

```

Golden tests are the only safety net

Because ODT is written from scratch with no PyTorch reference to compare against, every kernel needs a hand-computed expected value. A 2x3 matmul whose answer you worked out on paper is worth more than any amount of code review: it catches transposition errors, off-by-one indexing and wrong accumulator scales, all of which otherwise show up only as a model that trains slightly worse than it should — the hardest kind of bug to notice, let alone locate.

28.4 Profile Before Optimising

```

# Host profiling
gcc -pg prog.c -o prog && ./prog && gprof ./prog gmon.out

# Cache and branch behaviour
valgrind --tool=cachegrind ./prog
perf stat -e cache-misses,branch-misses ./prog

# On the MCU: DWT->CYCCNT around the region of interest

```

Intuition about performance is usually wrong

Programmers reliably guess the wrong hot spot. Measure first, optimise the measured bottleneck, then measure again to confirm the change helped. Optimising an unmeasured function is how code becomes unreadable without becoming faster.

28.5 Optimisations Worth Making

Technique	Typical gain	Note
Fix the loop order for cache locality	Up to several times	Innermost loop over the last index
Hoist loop-invariant work	10 to 30 percent	Especially divisions
Replace divide with reciprocal multiply	14x on that operation	Divide is 14 cycles, multiply is 1
Use the f-suffixed math functions	Up to 20x	Avoids double emulation
-O2 instead of -O0	2 to 5x	Free; just a flag
-Os for Flash-bound code	Smaller, sometimes faster	Less code means better I-cache hit rate
static inline for tiny hot functions	5 to 15 percent	Removes call overhead
SIMD / DSP intrinsics	2 to 4x	Last resort — hurts portability

```
/* Before: divide inside the loop, double-precision math */
for (size_t i = 0; i < n; i++) {
    out[i] = sqrt(in[i] / scale);
}

/* After: reciprocal hoisted, single-precision throughout */
const float inv = 1.0f / scale;
for (size_t i = 0; i < n; i++) {
    out[i] = sqrtf(in[i] * inv);
}
/* Roughly 20x faster on a Cortex-M7 for a 1152-element tensor. */
```

28.6 A Debugging Checklist

Symptom	Look at first
Wrong numbers, no crash	Uninitialised variable; integer division; wrong loop bound
Crash on some inputs only	Off-by-one; NULL not checked; buffer size
Works at -O0, fails at -O2	Undefined behaviour — run the sanitisers
Works on host, fails on MCU	Stack overflow; alignment; missing volatile; type width
Fails only after hours	Memory leak; counter overflow; heap fragmentation
Works in the debugger only	Race with an ISR; missing volatile; timing dependency

Exercises — Chapter 28

1. Compile an existing program with the full warning set and fix everything it reports.
2. Use gdb to set a breakpoint, inspect an array with the @ syntax, and print a backtrace.
3. Write a CHECK macro and a test suite for your bit-packing functions.
4. Write a golden test for a 2x3 by 3x2 matmul with hand-computed expected values.
5. Profile a program with gprof and identify the top three functions by time.
6. Take a loop containing a division and a sqrt and optimise both. Measure the improvement.
7. Find a bug that appears at -O2 but not -O0, and identify the undefined behaviour responsible.
8. Write a stack-usage check that fills the stack with a pattern at boot and reports the high-water mark later.

PART A

Exercise Solutions

Worked answers with the reasoning behind them, not just the result.

Chapter 1 — Getting Started

1. Two `printf` calls, each ending in `\n`. Compile with `gcc -Wall -std=c11 name.c -o name`.
2. GCC reports the error on the line *after* the missing semicolon, because the parser only discovers the problem when it meets a token that cannot continue the statement. Always check the line above the reported one.
3. Four windows: `4 * 1152 * sizeof(float) = 18432` bytes, or 18 KB. That is already 5.6 percent of the 320 KB SRAM budget for input data alone.
4. Around 800 to 900 lines. `stdio.h` pulls in a chain of other headers, and every declaration in all of them is pasted in. This is why compilation is slower than the source size suggests.
5. `./prog; echo $?` — the shell variable `$?` holds the exit status of the last command.

Chapter 2 — Variables, Types and Memory

1. On 64-bit Linux: `char` 1, `short` 2, `int` 4, `long` 8, `long long` 8, `float` 4, `double` 8, `size_t` 8. On Windows `long` is 4 — that is the one that surprises people and the reason to prefer `int64_t`.
2. 200 does not fit in `int8_t` (range -128..127). The conversion is implementation-defined and every real compiler wraps modulo 256, giving `200 - 256 = -56`.
3. Prints 0, meaning false. The usual arithmetic conversions turn -1 into unsigned `int` 4294967295, which is not less than 1.
4. `for (size_t i = n; i > 0; i--) { size_t idx = i - 1; ... }`. The loop variable never has to go below zero.
5. `float32` gives about 0.0010004640; `double` gives 0.0010000000. Only four digits agree. This is catastrophic cancellation, and it is exactly why ODT stores AdamW beta2 as a double.
6. `int callCount(void) { static int n = 0; return ++n; }`. The `static` gives it whole-program lifetime while keeping it invisible outside the function.
7. `010` is octal 8, so the result is 18. A leading zero changes the base.

Chapter 3 — Operators and Expressions

1. `17/5` is 3, times 5 is 15; `17%5` is 2; total 17. In general $(a/b)*b + a\%b == a$ for integers — that identity is what defines the remainder.
2. `(v >> 6) & 1u` or `(v & (1u << 6)) != 0`.
3. `uint8_t toggleBit(uint8_t v, int n) { return v ^ (uint8_t)(1u << n); }`
4. `==` binds tighter than `&`, so it parses as `x & (1 == 0) = x & 0 = 0`. Fix with `(x & 1) == 0`.
5. `+` binds tighter than `<<`, so it is `1 << 4 = 16`. For 9, write `(1 << 3) + 1`.
6. Both promote to `int`, giving 300. The cast back to `uint8_t` keeps only the low 8 bits: `300 - 256 = 44`.
7. `v = v < 0 ? 0 : (v > 255 ? 255 : v);`
8. See Chapter 17 for the full trace. The key points are converting global bit positions to positions within the byte, and that `value` holds 2^i at iteration `i` because the doubling happens after the test.

Chapter 4 — Control Flow: Branching

1. Guard style reads best: `if (n < 0) return NEG; if (n == 0) return ZERO; return POS;`
2. `n < 0 ? NEG : (n == 0 ? ZERO : POS)`. Acceptable at one level of nesting; beyond that `if/else` is clearer.
3. Six cases returning the six activity strings, plus a default returning `NULL`. Alternatively index a static `const char *[]` after a range check — usually better, since the data lives in Flash and no branching is needed.
4. Invert each condition, return the error immediately, and leave the success path unindented at the bottom. See section 4.3.
5. Execution falls through cases 1, 2, 3, 4 and 5 in order, running all five bodies. This is why `-Wimplicit-fallthrough` exists.

Chapter 5 — Control Flow: Loops

1. Accumulate into a `float` (or better, a `double` for the sum), then divide by 1152.0f.
2. The three forms are mechanically interchangeable; the `for` keeps the loop control on one line, which is why it is preferred for counted iteration.
3. $2 * (9 * 128) + 5 * 128 + 100 = 2304 + 640 + 100 = 3044$.
4. Nested loops `i` and `j` from 1 to 9, printing `%4d` and a newline after each row.
5. `size_t` is unsigned, so `i >= 0` is always true; when `i` wraps below zero it becomes `SIZE_MAX`. Use `for (size_t i = n; i > 0; i--)` with `i-1` as the index.
6. `rand() % (i+1)` is biased whenever `RAND_MAX + 1` is not divisible by `i+1`: the low residues occur one extra time. Rejection sampling, as in ODT's `rngBounded`, removes it.
7. Track `maxAbs` and its index, skipping any element where the mask bit is clear. Initialise `maxAbs` to a negative sentinel so the first active element always wins.

Chapter 6 — Functions

1. `void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }`, called as `swap(&x, &y)`.
2. Arguments are copied. The by-value version swaps the copies, which are discarded when the function returns. The caller sees nothing.
3. See section 8.5. Handle `n == 0` before touching `data[0]`.
4. Returning a struct is cleaner at the call site; out-parameters avoid a copy and let the caller reuse existing variables. For two floats either is fine; for a large struct, out-parameters win.
5. Recursive Fibonacci is exponential — roughly 2^n calls. Iterative is linear. At `n=35` the difference is seconds versus microseconds.
6. Loop over bytes, and for each byte over its 8 bits, or use `__builtin_popcount` per byte. Watch the final partial byte when `nBits` is not a multiple of 8.
7. $2048 * 4 = 8192$ bytes. That is the entire 8 KB stack with nothing left for the call frame itself — it will overflow. Make it `static` or take the buffer as a parameter.

Chapter 7 — Arrays

1. Nested loops with `m[i][j] = (int)(i*4 + j)`, printing with `%3d`.
2. Swap `arr[i]` with `arr[n-1-i]` for `i < n/2`.
3. $3 * 1152 + 7 * 128 + 45 = 3456 + 896 + 45 = 4397$.

4. Expect the row-major version to be roughly 3 to 10 times faster, depending on cache size. The gap widens as the matrix grows.
5. A simple accumulation loop. Consider a double accumulator to limit rounding error over long vectors.
6. Every byte becomes 0x01, so each float is the bit pattern 0x01010101, which is approximately 2.3694e-38. `memset` is only safe for zero.
7. Loop over `i`, call `tensorBoolGet(mask, i)`, and set `arr[i] = 0.0f` when it returns false.

Chapter 8 — Pointers: The Core Concept

1. Print with `%d`, `%p` (casting to `void*`) and `%d` again for `*p`. All three address prints match.
2. `void increment(int *p) { (*p)++; }`. The parentheses matter: `*p++` increments the pointer, not the value.
3. The `*` binds to the declarator, not the type. `int *a, b;` is `int *a` and `int b`. Write one declaration per line.
4. Return `bool`, write the index through a `size_t` * out-parameter only when found. The caller then cannot mistake a valid index 0 for "not found".
5. Both are errors. The first `const` makes the pointee immutable, the second makes the pointer itself immutable.
6. Switch on the tag and cast the `void *` to `int *` or `float *` accordingly. This is the tagged-union pattern done with a raw pointer.
7. The local's stack frame is reclaimed on return, so the pointer dangles. On an STM32 the memory is immediately reused by the next call, so the value appears correct for one or two instructions and then silently changes — far harder to diagnose than a segfault.

Chapter 9 — Pointers and Arrays Together

1. Inside `main` you get the full array size; inside the function you get 8 (or 4), because the parameter is a pointer. This is array decay.
2. `for (const float *p = data; p < data + n; p++) sum += *p;`
3. `p - a` is 3. `p[2]` is `a[5]`.
4. Declare `static const char *names[6]` and loop. Marking it `const` puts both the pointers and the literals in Flash.
5. `int *a[5]` is five pointers (40 bytes on 64-bit); `int (*a)[5]` is one pointer to a five-int array (8 bytes).
6. `for (r) for (c) data[r*cols + c] = 0.0f;` — or `memset`, since all-zero bytes are 0.0f.
7. `data + n` is legal to compute and compare. `*(data + n)` is undefined behaviour. The standard specifically permits one-past-the-end as an address so that loop conditions work.

Chapter 10 — Strings

1. `size_t n = 0; while (s[n]) n++; return n;`
2. Copy at most `size-1` bytes, then write `\0` unconditionally. Return the length that would have been needed, so the caller can detect truncation.
3. It compares addresses. It appears to work when the compiler merges identical string literals into one object, which it is permitted but not required to do.

4. Loop over the string, comparing each character against the five vowels in both cases — or use `strchr("aeiouAEIOU", c) != NULL`.
5. `strtok(buf, ",")` then repeated `strtok(NULL, ",")`, converting each token with `strtof`. Note that `strtok` modifies the buffer, so it cannot be a string literal.
6. It copies exactly `sizeof(buf)` characters and writes **no** terminator. Any subsequent `strlen` runs off the end. This is why `strncpy` is not the safe version of `strcpy`.
7. Range-check the index against 0..5 first, then index a static `const char *`. Return `NULL` outside the range.

Chapter 11 — Structures, Unions and Enums

1. `struct { float data[9][128]; int label; }` is $4608 + 4 = 4612$ bytes, and the compiler may pad to 4612 exactly since the alignment is 4.
2. Take a `const` pointer, loop over all 1152 values through `&s->data[0][0]`, and divide by 1152.0f.
3. 12 bytes: 1 for a, 3 padding, 4 for b, 1 for c, 3 tail padding. The tail padding keeps successive array elements aligned.
4. `int`, `char`, `char` gives 8 bytes: $4 + 1 + 1 + 2$ tail padding. A 33 percent saving.
5. 1.0f is 0x3F800000. On a little-endian machine the bytes print as 00 00 80 3F.
6. Enumerators for each type plus a trailing `Q_COUNT`, then `static const char *qnames[Q_COUNT] = { [Q_FLOAT32] = "float32", ... }`.
7. By value for two or three scalars, where the compiler keeps them in registers and the copy is free. By pointer for anything larger, and always by pointer in embedded code where stack is scarce.

Chapter 12 — Dynamic Memory

1. Check the return against `NULL` before use, and set the pointer to `NULL` after `free`.
2. The goto-cleanup pattern in section 12.2. Initialise every pointer to `NULL` so the cleanup block can free unconditionally — `free(NULL)` is defined to do nothing.
3. The second `free` of a live pointer corrupts the allocator's internal structures. After setting it to `NULL`, the second call is `free(NULL)`, which is a documented no-op.
4. Keep count and capacity. On overflow call `realloc` with double the capacity, and assign to a temporary first — if `realloc` fails it returns `NULL` and the original block is still valid, which assigning directly would leak.
5. Alternate 40-byte free gaps between used blocks: five gaps of 40 is 200 bytes free but no single gap fits 80.
6. `AddressSanitizer` prints the allocation stack trace and the number of bytes leaked, which usually identifies the line immediately.
7. Fragmentation would accumulate over thousands of steps until a tensor allocation fails, with no diagnostic and no recovery path on a device with no operating system.

Chapter 13 — File Input and Output

1. Read with `fgetc`s in a loop and count iterations. Note that a final line with no newline still counts.
2. Read into a buffer with `fread` and write however many bytes came back, until `fread` returns 0.
3. `fseek(f, 0, SEEK_END); long n = ftell(f); fseek(f, 0, SEEK_SET);`

4. `fseek(f, header + (long)i * 1152 * 4, SEEK_SET)` then `fread` 1152 floats. Check the return value.
5. Read six bytes and `memcmp` against `{0x93, 'N', 'U', 'M', 'P', 'Y'}`.
6. On Windows, text mode translates `0x0A` to `0x0D 0x0A` on write, corrupting any float whose bytes happen to include `0x0A`. The file then fails to load, usually on a different machine from the one that wrote it.
7. Open with mode `"a"`, which creates the file if absent and always writes at the end.

Chapter 14 — The Preprocessor

1. `#define MAX(a,b) ((a) > (b) ? (a) : (b))`
2. `SQUARE(1+2)` becomes `1+2*1+2 = 5` instead of 9.
3. `#define SWAP(a,b,T) do { T _t = (a); (a) = (b); (b) = _t; } while (0)`. The leading underscore reduces the chance of shadowing a caller variable.
4. `i++` appears twice in the expansion, so it may be evaluated once or twice. Use a static `inline` function, which evaluates each argument exactly once and type-checks them.
5. Use `#cond` to stringify the expression, plus `__FILE__`, `__LINE__` and `__func__`.
6. The second inclusion sees the guard macro already defined and expands to nothing.
7. The macro definitions themselves vanish from the output; only their expansions remain at the use sites.

Chapter 15 — Multi-File Programs

1. Prototypes and any shared types go in the header with include guards; the bodies go in the `.c` file; both `.c` files are compiled and linked together.
2. Declare a function in a header, call it, but never define it. The compile succeeds and the link fails.
3. Define `int counter`; in a header included by two `.c` files. Fix by making the header say `extern int counter`; and defining it once.
4. See section 15.4. Remember that Make requires tab characters, not spaces.
5. The other file gets "undefined reference" at link time, because the symbol has internal linkage and is not exported.
6. Compile errors concern one translation unit: syntax, types, unknown names. Link errors concern symbols across units: missing or duplicated definitions.
7. Guards make the repeated inclusion a no-op, so the types are defined exactly once.

Chapter 16 — The Standard Library

1. Call `qsort` with the comparator from section 16.3, then verify with a loop checking each element against its predecessor.
2. Casting the difference to `int` truncates any difference smaller than 1 to zero, so nearly-equal floats compare equal. For large values the subtraction can also overflow the `int` range.
3. Wrap each version in `clock()` calls and divide by `CLOCKS_PER_SEC`. Repeat enough iterations that the total is well above the clock resolution.
4. Expect a large speedup on any target without a double-precision FPU, and a measurable one even on hardware that has one.

5. `bsearch` requires the array to be sorted with a comparator consistent with the one passed to it. Mismatched comparators are a common silent bug.
6. Compile twice, once with and once without `-DNDEBUG`, and compare the disassembly or simply the behaviour on a failing input.
7. `FLT_EPSILON` is about $1.19\text{e-}7$. Use `fabsf(a-b) <= FLT_EPSILON * fmaxf(fabsf(a), fabsf(b))`.

Chapter 17 — Bit Manipulation

1. Test `(v >> n) & 1u`; set `v |= 1u << n`; clear `v &= ~(1u << n)`; toggle `v ^= 1u << n`.
2. $23 / 8 = 2$, $23 \% 8 = 7$ — bit 7, the most significant bit, of byte 2.
3. `startbitInternal = 2, endbitInternal = 6`. Bits 2,3,4,5 are set: $4+8+16+32 = 60 = 0b00111100$.
4. `mask = 0b00111100`; `0b11010110 & 0b00111100 = 0b00010100`; shift right by 2 gives `0b00000101 = 5`.
5. Clear the target bits with `data &= ~mask`, then OR in the shifted value: `data |= (value << startInternal) & mask`. The final AND guards against a value wider than the field.
6. Expect roughly an 8x speedup from `__builtin_popcount`, since it processes 32 bits per instruction rather than one bit per loop iteration.
7. For `v = 0b10110000`: `v-1 = 0b10101111`. The subtraction flips the lowest set bit to 0 and sets everything below it to 1, so the AND clears the whole low run, leaving `0b10100000`.
8. `#define CEIL_DIV(a,b) (((a) + (b) - 1) / (b))`. For 1000 bits: $(1000 + 7) / 8 = 125$ bytes.

Chapter 18 — Fixed-Point Arithmetic

1. `scale = 5.08 / 127 = 0.04`. Resolution 0.04, maximum rounding error 0.02.
2. $3.7 / 0.04 = 92.5$, rounds to 92 or 93; dequantised that is 3.68 or 3.72, so error 0.02. $-1.2 / 0.04 = -30$ exactly, error 0.
3. Divide, round, clamp to `[-128,127]` in a wider type, then narrow. The clamp must come before the cast.
4. $1024 * 127 * 127 = 16,516,096$. That is well within `int32`'s 2.1 billion — a margin of about 130x.
5. The empirical mean should land within about 0.002 of 0.3 for 100000 samples. The standard error is $\sqrt{p(1-p)/n}$.
6. $3.14159 * 256 = 804.24$, rounds to 804; $804 / 256 = 3.140625$. Error about 0.00097, which is a quarter of the 0.0039 resolution.
7. Any gradient below half an LSB rounds to zero every time, so the weight never moves. With many quantised layers and a small learning rate, most of the network enters this dead zone and training stalls.
8. `scale = 6.0 / 255 = 0.0235`, `zeroPoint = 0`. All 256 codes are used, whereas symmetric quantisation would waste the 128 negative ones.

Chapter 19 — Memory Layout

1. `const` table to `.rodata` (Flash); initialised global to `.data` (SRAM plus a Flash copy); uninitialised static array to `.bss` (SRAM only); function local to the stack; `malloc` result to the heap.
2. $5000 * 4 = 20000$ bytes of SRAM in `.bss`, with no Flash cost.
3. It moves to `.rodata`: 20000 bytes of Flash and zero SRAM. On a 320 KB budget that is a 6 percent saving for one keyword.

4. Use `size -A`. SRAM usage is `.data + .bss`; Flash is `.text + .rodata + .data`.
5. Weights 48 KB, gradients 48 KB, two AdamW moments 96 KB — 192 KB total, leaving about 128 KB for activations, stack and everything else. It fits, but not comfortably; plain SGD would free 96 KB of that.
6. $(12000 + 7) / 8 = 1501$ bytes. The mask is under one percent of the optimiser state — sparsity is essentially free in memory terms.
7. Uninitialised globals would contain whatever was in SRAM previously, so the C guarantee that they start at zero would no longer hold. Behaviour would differ between a cold boot and a reset.
8. There is nothing to return to. The startup code would continue executing whatever bytes follow the call, which is undefined. Hence the terminal for `(;;)`.

Chapter 20 — volatile, const and restrict

1. Without `volatile`, the compiler hoists the load out of the loop and tests a stale register value forever. At `-O0` it may appear to work, which makes the bug worse.
2. `const int *p`: the pointee is `const`. `int * const p`: the pointer is `const`. `const int * const p`: both.
3. Every pointer the function only reads through should be `const`. The compiler will tell you if you got it wrong.
4. `c++` is load, add, store. An interrupt between the load and the store, which also increments `c`, loses one increment. `volatile` guarantees the accesses happen but not that they are indivisible.
5. `#define REG (*(volatile uint32_t *)0x40021000)`, then `REG |= (1u<<3)` and `REG &= ~(1u<<3)`.
6. Calling `addArrays(buf, buf, n)` with `restrict` parameters is undefined: the compiler may have vectorised assuming no overlap, so the results will be wrong for the overlapping portion.
7. They constrain different parties. `const` restricts what the program may do; `volatile` describes what the hardware may do. A read-only status register is both.

Chapter 21 — Programming Without malloc

1. See section 21.3. Test with several alignments and confirm every returned pointer satisfies its requested alignment.
2. Adding `a-1` pushes past the next boundary, and `& ~(a-1)` clears the low bits to snap back down to it. It requires a power of two because only then is `a-1` a solid run of low bits.
3. Replace the `bool` array with a packed bitmap and scan for a clear bit — the same technique as the RgL mask.
4. Add a destination parameter and, if useful, a size parameter so the function can validate the buffer is large enough. Return a status code.
5. Weights 32 KB, gradients 32 KB, two moments 64 KB, mask 1000 bytes. Total about 129 KB.
6. The link error happens on the developer's machine, before the firmware ships. A `malloc` failure happens in the field, in a path nobody tested, possibly months later.
7. A tensor dimension read from a file, used directly as the `alloca` size. Prevent it by validating the value against a compile-time maximum before the allocation.

Chapter 22 — Function Pointers and Dispatch Tables

1. `float (*op)(float, float) = addFloats; float r = op(1.0f, 2.0f);`

2. The first is a function returning `int*`; the second is a pointer to a function returning `int`. `()` binds tighter than `*`.
3. A `static const` array of function pointers with designated initialisers, indexed by the enum. Being `const`, it lives in Flash.
4. Cast both `void *` arguments to the struct type, then return `(x > y) - (x < y)` on the field.
5. Unspecified entries are zero-filled, giving `NULL` function pointers you can check. An uninitialised array would contain garbage addresses, and calling one jumps into arbitrary memory.
6. Each handler returns the next state; the main loop is a single line indexing the table. Transition logic stays inside the handlers.
7. Compute element addresses as `(char *)arr + i * sz` and pass `ctx` straight through to the callback.
8. It decouples `executeOp` from every kernel signature. Adding a kernel with new parameters requires no change to `executeOp` at all.

Chapter 23 — Alignment, Padding and Endianness

1. `char a` at 0, 7 bytes padding, `double b` at 8, `char c` at 16, 1 byte padding, `short d` at 18, 6 bytes tail padding. Total 24.
2. `double`, `short`, `char`, `char` gives 16 bytes: $8 + 2 + 1 + 1 + 4$ tail. A saving of 8 bytes, or 33 percent.
3. Cast the address to `unsigned char *` and print four bytes. On x86 or ARM you will see 78 56 34 12.
4. See section 23.4. Test with a known byte sequence and confirm the value, then round-trip it.
5. The padding bytes are uninitialised and hold leftover stack or heap contents. Two structs with identical members can differ in padding.
6. `struct __attribute__((packed)) { uint8_t magic[4]; uint32_t version; uint32_t layerCount; }` is exactly 12 bytes.
7. The address may not satisfy the type's alignment requirement. Dereferencing it can raise a `UsageFault` on ARM, and passing it to a function expecting alignment is undefined behaviour.
8. Same structure as `readLE32` but with the shift amounts reversed: `(p[0]<<24) | (p[1]<<16) | (p[2]<<8) | p[3]`.

Chapter 24 — Overflow and Undefined Behaviour

1. At `-O0` it usually wraps as the hardware does. At `-O2` the compiler may assume overflow is impossible and eliminate the surrounding check entirely, producing different output.
2. Unsigned overflow is fully defined to wrap modulo 2^N , so the result is identical at every optimisation level.
3. Check before adding, as in section 24.2, or use `__builtin_add_overflow`, which maps to a single instruction plus a branch.
4. The condition can only be true if signed overflow occurred; overflow is undefined; therefore the compiler may assume the condition is always false and delete it.
5. Cast a `float*` to `int*` and dereference. Fix with `memcpy`, which the optimiser reduces to the same single move instruction.
6. ASan reports the exact line, the size of the object, and how far past the end the access went.
7. Signed overflow, shift by too large an amount, out-of-bounds access, use after free, and strict aliasing. Defences: unsigned types or overflow builtins, bounds checks on shift amounts, length parameters, `NULL` after

free, memcpy for type punning.

8. 1000 is outside int8_t's range, and out-of-range float-to-integer conversion is undefined. Clamp in a wider integer type first, then narrow.

Chapter 25 — Floating Point on Microcontrollers

1. 1.0f is 0x3F800000 (S=0, exp=127, mantissa=0); 0.5f is 0x3F000000 (exp=126); -2.0f is 0xC0000000 (S=1, exp=128).
2. The sum is 0.30000001192092896f. Neither operand is exactly representable in binary.
3. Naive float32 summation drifts substantially; Kahan summation stays close to the exact 1000000.0 for the cost of three extra operations per element.
4. Relative tolerance scales with magnitude, so the same function works near 1.0 and near 1e6, where an absolute tolerance would fail at one end or the other.
5. $2^{24} = 16777216.0f$. Beyond that the spacing between representable floats exceeds 1, so adding 1 changes nothing.
6. Expect roughly a 10x improvement, since divide is about 14 cycles and multiply is 1.
7. The betas are scalars, computed once per step, so the cost of double is negligible — while 1 - beta² in float32 loses three decimal places, and that error is then compounded thousands of times through the bias correction. The weights, by contrast, are millions of elements in the inner loop, where double would be ruinous.
8. `bool isNan(float x) { return x != x; }` — NaN is the only value that compares unequal to itself.

Chapter 26 — Binary Formats and Serialisation

1. Build the 12 bytes explicitly with writeLE32 and compare after reading back.
2. Read the version field and return a distinct error code for a known-but-old version, so the caller can print something useful rather than "corrupt file".
3. Write tag, then length, then payload. On read, always consume exactly length bytes, using fseek for unknown tags.
4. Write the count as LE32, then each float via writeLE32 of its bit pattern obtained with memcpy — not by casting a float* to uint32_t*.
5. Write (numWeights + 7) / 8 bytes of packed mask under a new tag, preceded by the weight count so the reader can validate.
6. Read the count, compute the byte length, read the bytes, and compare bit-by-bit against the original with tensorBoolGet.
7. Padding differs between compilers and optimisation levels; a file written by an x86 debug build may not be readable by an ARM release build of the same source.
8. Dimension count against MAX_DIMS, each dimension against a maximum, the product against a maximum byte count, the layer count against MAX_LAYERS, and every TLV length against the remaining file size.

Chapter 27 — ARM Cortex-M Specifics

1. `#define REG (*(volatile uint32_t*)0x40021000u)` plus the standard set, clear and test idioms on bit 5.

2. The hardware changes them independently of the CPU, and some reads have side effects such as clearing a status flag. Without volatile the compiler may cache, merge or delete accesses.
3. The ISR clears its flag and increments a volatile `uint32_t`. The main loop reads it inside a critical section if it needs a consistent multi-word view.
4. It is a read-modify-write; an ISR writing the same register between the read and the write is lost. BSRR takes a bitmask and sets or clears atomically, with no read.
5. Save PRIMASK, disable, do the work, restore. Restoring preserves the state of any enclosing critical section, which unconditional enabling would destroy.
6. Enable TRCENA, zero CYCCNT, enable CYCCNTENA, then read before and after.
7. Both operands are unsigned, and unsigned subtraction is defined modulo 2^{32} , so the difference is correct as long as fewer than 2^{32} cycles elapsed.
8. Clean before DMA reads memory the CPU wrote; invalidate before the CPU reads memory DMA wrote. Buffers must be 32-byte aligned so the operations do not affect neighbouring data.

Chapter 28 — Debugging, Testing and Optimisation

1. Expect `-Wconversion` to produce the most output on existing code; most findings are genuine narrowing that should be made explicit.
2. `break, run, print arr[0]@10, bt.`
3. The CHECK macro from section 28.3, with one test function per behaviour and a summary count in main.
4. Compute the six output values by hand, then CHECK_NEAR each against the kernel output with a tolerance of about $1e-5$.
5. gprof reports a flat profile sorted by self time; the top entry is where to start.
6. Hoist the reciprocal and switch to `sqrtf`. Expect roughly an order of magnitude on a target without a double FPU.
7. Signed overflow and strict-aliasing violations are the two most common causes of optimisation-level-dependent behaviour. Run `-fsanitize=undefined` to identify which.
8. Fill the unused stack region with a known pattern such as 0xAA at boot, then later scan upward for the first byte that still matches to find the high-water mark.

PART B

Quick Reference

Tables to look things up in once you know what you are looking for.

B.1 Fixed-Width Types

Type	Bytes	Range	printf
int8_t	1	-128 .. 127	%d
uint8_t	1	0 .. 255	%u
int16_t	2	-32768 .. 32767	%d
uint16_t	2	0 .. 65535	%u
int32_t	4	-2147483648 .. 2147483647	%d or PRId32
uint32_t	4	0 .. 4294967295	%u or PRIu32
int64_t	8	about +/- 9.2e18	PRId64
size_t	4/8	0 .. SIZE_MAX	%zu
ptrdiff_t	4/8	signed	%td
float	4	about +/- 3.4e38, 7 digits	%f or %e
double	8	about +/- 1.8e308, 15 digits	%f or %e

B.2 Operator Precedence, Highest to Lowest

Operators	Associativity
() [] -> . ++(post) --(post)	Left to right
! ~ ++(pre) --(pre) + - * & (cast) sizeof	Right to left
* / %	Left to right
+ -	Left to right
<< >>	Left to right
< <= > >=	Left to right
== !=	Left to right
&	Left to right
^	Left to right
	Left to right
&&	Left to right
	Left to right
?:	Right to left
= += -= *= /= %= &= = ^= <<= >>=	Right to left
,	Left to right

B.3 Bit Idioms

Goal	Expression
Test bit n	<code>(v >> n) & 1u</code>
Set bit n	<code>v = (1u << n)</code>
Clear bit n	<code>v &= ~(1u << n)</code>
Toggle bit n	<code>v ^= (1u << n)</code>
Clear lowest set bit	<code>v & (v - 1)</code>
Isolate lowest set bit	<code>v & -v</code>
Is power of two	<code>v && !(v & (v - 1))</code>
Modulo a power of two	<code>v & (n - 1)</code>
Round up to multiple of n	<code>(v + n - 1) & ~(n - 1)</code>
Ceiling division	<code>(a + b - 1) / b</code>
Bytes for n packed bits	<code>(n + 7) / 8</code>
Byte index of bit i	<code>i / 8</code>
Bit index within byte	<code>i % 8</code>
Population count	<code>__builtin_popcount(v)</code>

B.4 printf Format Specifiers

Specifier	Argument type
<code>%d %i</code>	int
<code>%u</code>	unsigned int
<code>%ld %lu</code>	long, unsigned long
<code>%lld %llu</code>	long long
<code>%zu</code>	size_t
<code>%td</code>	ptrdiff_t
<code>%f %.2f</code>	double (float is promoted)
<code>%e %g</code>	double, scientific / shortest
<code>%s</code>	char *
<code>%c</code>	int, printed as a character
<code>%p</code>	void * — cast explicitly
<code>%x %X</code>	unsigned, hex
<code>%08X</code>	unsigned, hex, zero-padded to 8
<code>%%</code>	a literal percent sign

B.5 Compiler Flags

Flag	Effect
-Wall -Wextra	Essential warnings
-Werror	Warnings become errors
-Wconversion	Warn on implicit narrowing
-std=c11	Select the language standard
-g	Emit debug symbols
-O0 / -O2 / -Os	None / speed / size
-fsanitize=address	Runtime memory-error detection
-fsanitize=undefined	Runtime UB detection
-E	Stop after preprocessing
-S	Stop after generating assembly
-c	Compile to an object file, do not link
-lm	Link the math library
-DNAME=value	Define a preprocessor macro
-Ipath	Add an include search directory

B.6 Cortex-M7 Operation Costs

Operation	Approximate cycles
Integer add, subtract, bitwise	1
Integer multiply	1 to 2
Integer divide	2 to 12 (variable)
float add, subtract, multiply	1
float fused multiply-accumulate	1
float divide	14
float square root	14
expf, logf, sinf (library)	30 to 100
double operations	Software emulated — tens of cycles
SRAM access (cached)	1
Flash access (ART hit)	1 to 2
Branch, mispredicted	3 to 5

B.7 Common Undefined Behaviours

Undefined behaviour	Safe alternative
Signed integer overflow	Unsigned types, or <code>__builtin__*_overflow</code>
Shift by $\geq$ type width	Check the shift amount first
Out-of-bounds array access	Carry the length; check before indexing
Dereferencing NULL	Check every pointer parameter
Use after free	Set the pointer to NULL immediately after free
Reading an uninitialised variable	Always initialise at the declaration
Strict-aliasing violation	<code>memcpy</code> , or a union
Out-of-range float to int	Clamp in a wider integer type first
Modifying a string literal	Declare it <code>const char *</code>
Two modifications in one expression	Split into separate statements

B.8 HAR Dataset Geometry

Quantity	Value
Sampling rate	50 Hz
Window length	128 samples = 2.56 seconds
Channels	9 (3 accel + 3 gyro + 3 body accel)
Values per window	1152
Bytes per window (float32)	4608
Classes	6: walking, upstairs, downstairs, sitting, standing, laying
Training samples	6617
Validation samples	735
Test samples	2947
Full training set size	about 30.5 MB — 95x the available SRAM

B.9 STM32 Nucleo-F746ZG

Resource	Value
Core	ARM Cortex-M7, ARMv7-M
Clock	up to 216 MHz
Flash	1 MB
SRAM total	320 KB (240 KB SRAM1 + 16 KB SRAM2 + 64 KB DTCM)
L1 cache	4 KB instruction + 4 KB data
FPU	Single precision, FPUv5-SP-D16

Resource	Value
DSP	SMLAD, SMUAD, QADD8, SSAT and related
MPU	8 regions; no MMU, no virtual memory
Endianness	Little-endian